

פרויקט במערכות מבוזרות – 02360351

לירז כליף

ת.ז : 211765391

תוכן עניינים

4	 ארכיטקטורת המערכת :
4	 עץ המערכת :
4	 רכיבי המערכת :
5	 פרוטוקולי תקשורת :
5	 logging and snapshots – שמירת מידע :
6	 Paxos gRPC and Protocol
6	 מבנה הנתונים של <i>gRPC Protobuf Messages</i>
6	 מבנה הנתונים של <i>PaxosInstance</i>
7	 פרוטוקול Paxos
8	 Adaptive Quorumn מנגנון
9	 Failure Detection and Leader Election
9	 בחירת מנהיגות וזיהוי כשלים
9	 מבנה הנתונים של ElectionManager :
10	 מנגנון זיהוי הכשלים
10	 תרחישי הכשל והטיפול בהם
10	 מנגנון בחירת המנהיג
11	 Multi-Paxos אופטימיזציות
11	 Membership
11	 מבנה הנתונים של MembershipManager :
12	 מנגנון הפעולה
13	 Consensus Messages for Multiple Instances – Multi-Paxos
13	 הרחבה של מבנה הנתונים של <i>gRPC Protobuf Messages</i>
13	 מבנה הנתונים MultiPaxosInstance
14	 מבנה הנתונים PaxosServer
14	 פרוטוקול Multi-Paxos
14	 ניהול בקשות לקוח בעזרת MultiPaxos
15	 מקביליות הודעות – Pipeline
16	 קריאות ליניאריות
16	 Replicated Log and State Machine
16	 מנגנוני התקשורת במערכת
16	 מכונת המצבים – Auction State Machine
16	 מבני הנתונים הבסיסיים של המערכת
17	 מבנה הנתונים של ה- State Machine
18	 עיבוד בקשה במכונת המצבים
19	 פקודות מכונת המצבים והלוגיקה של המערכת
19	 Node Recovery and Synchronization
20	 הרחבה של מבנה הנתונים של <i>gRPC Protobuf Messages</i>
21	 Snapshots and Consensus Disposal
21	 הרחבה של מבנה הנתונים של <i>gRPC Protobuf Messages</i>
22	 מבנה הנתונים של SnapshotData
22	 יצירה ושמירה של Snapshot
23	 גיוסם הלוגים – Safe Log Compaction
24	 מנגנון הורדת תמונת המצב למערכת – InstallSnapshot
24	 Data Storage - WAL
24	 מבנה הנתונים של WALManager
25	 שמירה לדיסק – PersistState

25.....	אינטגרציה של WAL עם PaxosServer
26.....	Client API
26.....	ניהול (HTTP Server) interfacen
26.....	בקשות הכתיבה Linearizability Write
26.....	בקשות הקריאה הליניארזיביליות
26.....	בקשות הקריאה הסדרתיות
26.....	Batching API
27.....	Linearizable Write Endpoints RESTful API Endpoints
27.....	Linearizable Reading Endpoints
27.....	Sequential Reading Endpoints
28.....	Sequential System Reading Endpoints
28.....	Docker
28.....	תהליך הבנייה Docker file
28.....	ארכיטקטורת הפריסה Docker Compose
28.....	שער הכניסה – NGINX Gateway
29.....	Testing and Client Demo
29.....	תהליך הבנייה וקמפול המערכת
30.....	הוכחת Membership
30.....	עדכון טופולוגית ה-quorum Dynamic
30.....	התאוששות וסנכרון לוגים
30.....	Client Demo
30.....	מבחן 1 – מבחן שפיות בסיסי
31.....	מבחן 2 – Batching Request
32.....	מבחן 3 – עקביות
32.....	מבחן 4 – נפילת מנהיג ו-recovery
33.....	מבחן 5 – נפילת follower, סנכרון של השרת בעליה והשמת snapshot
34.....	מבחן 6 – מבחן עומס על המערכת – 1000 בקשות במקביל
34.....	מבחן 7 – מקסימום נפילות שרתים
36.....	מבחן 8 – מבחן העברת הודעות מעוקב למנהיג
36.....	מבחן 9 – קריאה סדרתית
36.....	מבחן 10 – השלמת פעולות ה-API במערכת – הוכחה שלהן
37.....	UI - Application
40.....	סיכום ומסקנות

ארכיטקטורת המערכת :

המערכת המתוכננת היא מערכת מכרזים מבוזרת. המערכת מבוססת על אלגוריתם Multi-Paxos לניהול קונצנזוס ועקביות נתונים. המערכת מנוהלת בסביבת docker מרובת קונטיינרים, בסביבה של 12 צמתים. המערכת משתמשת ב etcd עבור בחירת מנהיג. המערכת מתוכננת לעמוד בנפילות וביצועים גבוהים תחת עומס, ומשתמשת ב log וב snapshots על מנת לשמור את המידע של המערכת.

עץ המערכת :

המערכת המבוזרת נמצאת בתוך src/server/ ובנויה בצורה הבאה :

```
C: .
├── go.mod
├── go.sum
├── main.go
├── api
│   ├── auction_api.go
│   ├── log_filter.go
│   └── server.go
├── election
│   └── election.go
├── logger
│   └── logger.go
├── membership
│   └── membership.go
├── paxos
│   └── paxos.go
├── proto
│   ├── paxos.pb.go
│   ├── paxos.proto
│   └── paxos_grpc.pb.go
├── state
│   └── auction.go
└── storage
    └── wal.go
```

קבצי ה docker נמצאים ב src/docker :

```
C: .
├── docker-compose.yaml
├── Dockerfile
└── nginx.conf
```

רכיבי המערכת :

בנינו את המערכת בארכיטקטורת microservices עם הרכיבים הבאים :

- Cluster של 12 שרתי Paxos זהים server0-server11. כל שרת מריץ במקביל :
 - API layer : שכבת HTTP המטפלת בבקשות לקוח.
 - Consensus layer : שכבת Multi-Paxos המנהלת את הנתונים ואת הקונצנזוס של המערכת
 - State Machine : שכבת הלוגיקה של המערכת הספציפית שלנו – לוגיקה של מערכת המכרזים, הפקודות והקריאות האפשריות.
- 2. Coordination service – etcd : שירות עבור בחירת המנהיג וניהול הכשלים.
- 3. Gateway – Nginx : שימוש ב docker ב reserve proxy , על מנת לנתב בקשות של לקוחות אל צמתים ב cluster בשיטת round robin. זאת על מנת ליצור מערכת שקופה עבור הלקוח של פנייה לשרתים ויצירה של פנייה אחידה לכל המערכת.
- 4. Membership

פרוטוקולי תקשורת :

- תקשורת פנימית בין שרתים (node to node) : תקשורת gRPC. כל ההודעות הפנימיות מתנהלות ע"י תקשורת זו : prepare, accept, commit, synclog.
- תקשורת שרת-לקוח : תקשורת בין הcluster אל הלקוח מתנהל בעזרת תקשורת http/REST. הלקוחות פונים לגateway והשרתים מחזירים תשובת json. נוסף מנגנון פנימי של batching שמאפשר שליחת רשימת פקודות בבקשת http יחידה לשיפור ביצועים.

שמירת מידע – logging and snapshots :

- Wal – מתבצע שמירה לדיסק
- Snapshots – מופעל מנגנונים לשמירת תמונת מצב ומחיקת לוגים ישנים

קבצי המערכת :

מטרה	קובץ
Entry point, parses flags, initializes server	main.go
PaxosServer implementation, gRPC handlers, consensus logic	server/api/server.go
Paxos instance state, quorum logic, phase handlers	server/paxos/paxos.go
Leader election via etcd, campaign and observation	server/election/election.go
Dynamic membership tracking via etcd watches	server/membership/membership.go
State machine logic, validation, command handlers	server/state/auction.go
RESTful API endpoints, HTTP handlers	server/api/auction_api.go
Write-ahead logging for crash recovery	server/storage/wal.go
Protobuf definitions for gRPC messages	server/proto/paxos.proto
SSE (server sent events) handler, streams live logs to frontend	server/api/log_filter.go
Event hub, collect system events for monitoring	server/logger/logger.go
Container build configuration	docker/Dockerfile
Multi-container orchestration	docker/docker-compose.yaml
Reverse proxy configuration, gateway routing	docker/nginx.conf

Paxos gRPC and Protocol

בפרק זה נתאר את האלגוריתם הבסיסי של הבטחת הקונצנזוס. המימוש מבוצע במחלקה *PaxosInstance*, שהינה יחידת הסכמה יחידה אטומית ועצמאית. נראה כי באופן כללי מנגנון Paxos אחראי על ניתוב הבקשות בצורה עקבית, ומשתמש בשתי ממשקי תקשורת, גם כאשר הלקוח פונה לשרת שאינו המנהיג.

כפי שתיארנו, מנגנון PaxosServer בעל שני ממשקי הקשבה של תקשורת. הקומפוננט *MultiPaxosInstance*, שמכילה את ערכי *PaxosInstance* היחידים, מדברת יחד עם צמתים אחרים במערכת בעזרת מנגנון תקשורת ה-gRPC: gRPC פנימי – הינו ממשק RPC בינארי, המאזין בפורט נפרד. משמש לתקשורת מהירה ויעילה בין הצמתים ב-cluster. השימוש ב-protobuf מבטיח סריאליזציה יעילה ומהירה של הודעות פנימיות של המנגנון עצמו (כמו Prepare, Accept, Commit).

מבנה הנתונים של gRPC Protobuf Messages

הודעות הנתונים מוגדרות בקבצי ה-proto וכוללות את השדות:

- **PrepareRequest**: הודעת Prepare מהמנהיג לצמתים.
 - InstanceId - האינדקס בלוג עליו מנסים להסכים
 - ProposalId - ה-ID איתו מנהיג מציע את עצמו בתור מנהיג. חייב להיות גדול יותר מכל מה שהצמתים ראו בעבר.
 - SenderId – מזהה של השרת השולח
- **PromiseResponse**: הודעת הבטחה מהצמתים למנהיג.
 - Promised – משתנה בוליאני, האם המנהיג התקבל או נדחה
 - AcceptedValue/ AcceptedId – משתנים שנועדו להבטיח correctness. משתנים שערכם הינו על איזה ערך ואיזה ID הצומת הצביע בעבר. בעזרת משתנים אלו המנהיג בוחר את הערך עם ה-ID הגבוה ביותר ואינו דורס החלטות קודמות.
- **AcceptedRequest**: הודעת בקשה מהמנהיג לצמתים, עבור ערך מסוים
 - Value : המידע שהמנהיג מציע
 - InstanceId : האינדקס בלוג אליו שייך ה-value
 - ProposalId : מזהה לאימות Promise
 - SenderId : המזהה של הצומת השולח
- **CommitRequest**: הודעת החלטה סופית עם הערך הנבחר. מכילה:
 - Value : הערך הסופי שהמנהיג שולח לעדכון הצמתים.
 - InstanceId : האינדקס בלוג שיש לסמן בו Committed
 - SenderId : מזהה של המנהיג השולח.

מבנה הנתונים של PaxosInstance

מופע Paxos יחיד מנהל state machine פנימי ששומר על safety ועל liveness. מנהלת הסכמה על אינדקס ספציפי (i). האינדקס מייצג את המקום הסידורי של הפקודה הספציפית בתוך הלוג הגלובלי המשוכלל.

- **InstanceId (int32)**: מזהה חד ערכי של מיקום ההחלטה בלוג. משנה זה מאפשר למערכת לנהל החלטות במקביל, אך עדיין ליישם אותם בצורה סדרתית ולשמור על ליניארזיביליות במכונת המצבים.
- **MaxPromisedID (int64)**: שדה קריטי עבור safety, משמש כשומר הסף. משתמש זה שומר את מספר ההצעה הגבוה ביותר שהצומת נתקל בו. הצומת דוחה הודעות Prepare, Accept

בעלות מזהה נמוך יותר מערך זה. בעזרת משנה זה, מנהיגים ישנים אינם יכולים לשבש החלטות של המנהיג העדכני.

- **AccpetedID (int64)** : מציין את מזהה ההצעה של הערך השמור בצומת. אם מנהיג חדש מנסה להציע את עצמו ולהשתלט על האינדקס, הוא חייב לדעת עם הערך השמור בצומת *AcceptedValue* הוא עדכני או לא כדי להחליט אם לאמץ אותו.
- **AcceptedValue ([byte])** : הערך שעליו מנסים להסכים (במערכת שלנו batch של פקודות json). במידה ויש הסכמה של quorum על הערך המועמד, המנהיג יצטרך לאמץ את הערך הזה.
- **IsCommitted (bool)** : דגל שנועד לייצג את ההחלטה והאימוץ של הערך המוצע. מסמן שהערך המוצע אושר ע"י quorum, ברגע שמסומן בtrue עבור ערך מסוים, מוכן להעביר לשכבת האפליקציה (Apply).
- **Promises (map[int32]bool)** : מפה לשימוש המנהיג בלבד, העוקבת אחר תשובות Prepare משאר הצמתים בקלאסטר. המפתח של המפה הוא ה-ID של כל שרת בקלאסטר, והערך הוא אם אושר או לא אושר ההבטחה. בעזרת מפה זו מחושב quorum בשלב 1.
- **Accepts (map[int32]bool)** : מפה לשימוש המנהיג בלבד, בעוקבת אחר תשובות Accept משאר הצמתים בקלאסטר. המפתח של המפה הוא ה-ID של כל שרת בקלאסטר, והערך הוא אם התקבל או לא. בעזרת מפה זו מחושב ה-quorum בשלב 2.

פרוטוקול Paxos

הפרוטוקול מובל על ידי המנהיג (proposer) ומבוצע אל מול כלל הצמתים ב-cluster (acceptors).

שלב 1 – Prepare and Promise

בשלב זה המנהיג מוודא כי הנתון שהוא מתכוון להציע הוא העדכני ביותר, וכי לא התקבלה החלטה קודמת.

המנהיג : proposer מייצר מזהה ייחודי להצעה *ProposalID*. המזהה מורכב ממונה מונוטוני (counter) וה-ID של השרת, על מנת להבטיח ייחודיות וסדר כרונולוגי. ה-proposer שולח הודעת *Prepare(ProposalID)* לכל שאר הצמתים.

העוקבים : העוקבים מקבלים את ההודעה ומפעילים את *Prepare_Instance*. במידה והמזהה הוא העדכני ביותר, הצומת מעדכן את ה-*MaxPromisedID* לערך החדש (*ProposalID*). הצומת מחזיר למנהיג הודעת הבטחה *Promised = True* ואם הערך כבר קיים את ה-*AcceptedValue* שהינו בעל מזהה גבוה יותר. במידה והמזהה אינו העדכני ביותר, (לדוגמא במקרה של מנהיג ישן שהופעל מחדש), הצומת דוחה את ההצעה ומחזירה *Promised = False*.

שלב 2 – Propose and Accept

מטרת שלב זה היא להשיג קונצנזוס.

המנהיג : אוסף את תשובות ה-*Promise* מכלל הצמתים. מבצע בדיקת קוורום : בודק אם יש לו רוב לפי הלוגיקה לפי כמות השרתים (*Adaptive quorum*). מבצע בחירת ערך : המנהיג בודק אם מישחו מהעוקבים דיווח על *AcceptedValue* קיים. אם כן : המנהיג זונח את הערך המקורי ומאמץ את הערך בעל ה-ID הגבוה ביותר שדווח לו. כך בעצם אנו מונעים שכתוב של ההיסטוריה. אם לא : המנהיג מציע את הערך שלו ושולח הודעת *Accept(ProposalID, Value)* לשאר הצמתים.

העוקבים : כל צומת מקבל הודעת *Accept* ומפעיל את *Accept_Instance*, בצורה הבאה :

נוודא שוב כי ה-*ProposalID* הינו הגבוה ביותר בצומת. שלב זה חשוב למקרה שבין שלב 1 לשלב 2, מנהיג עם *ID* גבוה יותר שלח בקשת *Prepare* קודם ונעשה המנהיג. בדיקה זו מונעת מצב של דריסה של המנהיג הישן על המנהיג הנוכחי. במידה ותנאי זה מתקיים הצומת שומר את הערך של $AcceptedID = ProposalID, AcceptedValue = Value$. הוא כותב אותו אל *WAL* (עבור שרידות) ומחזיר אישור $Accepted = True$.

שלב 3 – Commit

המנהיג סופר את כמות הודעות ה-*Accepted* שהתקבלו. בעזרת הפונקציה *HasWriteQuorum* אנחנו בודקים ב-*cluster* אם קיים *quorum* של כתיבה על לוגיקת ה-*quorum* לפי מספר הצמתים. עבור הדוגמא שאנחנו בנינו, *cluster* עם 12 צמתים אנחנו נבדוק *grid quorum*. אם קיים *quorum* נעל את הערך והמנהיג יסמן $IsCommitted = True$, נשלח הודעת *Commit* אסינכרונית לשאר הצמתים. כל צומת שתקבל את הודעת ה-*Commit* תעדכן את המופע המקומי שלו ויסמן אותו כסופי ויעביר אותו לשכבת האפליקציה (*applyLog*).

מנגנון Adaptive Quorum

במערכת זו בחרנו לממש 3 שיטות לקבלת *quorum* ע"פ כמות השרתים במערכת. ככל שה-*cluster* גדול יותר, כך עומס תעבורת המידע גבוה יותר. לכן, על מנת לעמוד בדרישות הביצועים ולייעל תעבורה ברשת, מימשנו מנגנון *Adaptive Quorum* אשר מרחיב את הפרוטוקול הבסיסי ומאפשר גמישות וביצועים משופרים.

האסטרטגיה של ה-*quorum* מתבצעת בזמן אמת על פי כמות השרתים, ונקבעת ע"י פונקציית *GetQuorumType* בצורה הבאה:

- עד 5 צמתים: נשתמש ב-*majority* רגיל עבור *quorum*.
 - עבור 6-10 צמתים: נשתמש ב-*asymmetric quorum*.
 - מעל 10 צמתים: נשתמש ב-*grid quorum*, או *bi-quorum*.
- לוגיקת ההצבעה של ה-*grid quorum* עובדת בצורה הבאה:

WriteQuorum – על מנת לאשר את הכתיבה נשתמש ב-*HasWriteQuorum*, המנהיג צריך לקבל אישור משורה מלאה של צמתים.

Election/Read Quorum – כדי להיבחר להיות מנהיג או לקרוא מתוך נשתמש ב-*HasElectionQuorum*, כך שנדרש נציג מכל שורה.

תכונת המטריצה מבטיחה לנו כי תמיד יהיה צומת אחד משותף בין כל שורה מלאה לבין כל קבוצה המכילה נציג מכל שורה. צומת החיתוך מבטיחה את תקינות הפרוטוקול.

חשיבה ארכיטקטונית:

בחרנו במנגנון אדפטיבי על מנת לבצע *scaling* יעיל במערכת. ב-*cluster* שלנו – 12 צמתים, כתיבה דורשת אישור מ-4 צמתים בלבד (במקום 7 ברוב רגיל). בחירה זו מקטינה את ה-*latency* ומשפרת את ה-*throughput* של המערכת תחת עומס כבד, תוך עמידה ב-*multi consensus*.

השימוש ב-*majority* עבור קלאסטר קטן, הינה בשביל לבצע אופטימיזציה לשרידות המערכת. בקלאסטר קטן, צוואר הרשת אינה צוואר בקבוק ושימוש במנגנון זה נותן לנו עמידות גבוהה יותר של המערכת. עבור קלאסטר בינוני, אנחנו בעצם מבצעים אופטימיזציה של המקרה השכיח. אנחנו מבצעים הקטנה של הרוב הדרוש לכתיבה ל-3 צמתים קבועים, מה שנותן לנו זמן תגובה מהיר מאוד. מצד שני אנחנו מגדילים את הרוב הדרוש לבחירות *Election Quorum*. מכיוון שההנחה היא כי רוב הזמן המערכת מבצעת יותר בקשות כתיבה תחת מנהיג קיים, וכי אנו מסתמכים על *etcd failure detector* עבור ניהול כשלים ובחירת מנהיג, כך שניהול הצמתים שלנו מבוצע בצורה

דינמית במערכת, מה שעוזר למנוע *deadlock* גם במקרה של נפילת שרתים. ומנגנון *quorum* יתבסס על הרשימה העדכנית. עבור קלאסטר גדול נרצה לבצע אופטימיזציה לתעבורה. בקלאסטר גדול, המתנה לרוב רגיל הינה צוואר בקבוק עבור התעבורה של הרשת. שימוש ב*grid quorum* דורשת אישור משורה אחת של צמתים (עבור 12 צמתים בקלאסטר – דורשת 4 צמתים), ומקטינה את הסיבוכיות מ $O(N)$ אל $O(\sqrt{N})$, תוך הבטחה על חיתוך מתמטי.

Failure Detection and Leader Election

במערכת מבוזרת, מימוש זיהוי הנפילות ובחירת מנהיג הם תנאים הכרחיים לiveness של הפרוטוקול. במערכת הנוכחית השתמשנו ב*etcd* עבור מימושים אלה. כך ש-*etcd* משמש כמקור אמת יחידי עבור סטטוס הצמתים והמנהיגות.

המחלקה *ElectionManager* ב*election.go*, מכילה את המנגנון האחראי להבטחת מנהיג אחד ויחיד בכל רגע נתון, עבור *safety*, ובחירת מנהיג חדש במהירות במקרה של נפילת המנהיג, ושמירה על הiveness.

חשיבה ארכיטקטונית – שימוש ב*etcd*:

הבחירה להשתמש ברכיב חיצוני, *etcd*, עבור זיהוי כשלים ובחירת מנהיג עוזר לפשט את הפרוטוקול, לנהל את הiveness של Paxos וחוסך את הצורך במימוש מנגנוני Heartbeat פנימיים.

בחירת מנהיגות וזיהוי כשלים

מבנה הנתונים של ElectionManager:

- *EtcdClient (* clientv3. Client)*: אובייקט המחזיק את החיבור הפיזי (TCP) לקלאסטר *etcd* ודרכו מתבצעות כל פעולות הרשת.
- *Session (* concurrency. Session)*: אובייקט קריטי עבור מנגנון זיהוי הכשלים. אובייקט זה מאגד בתוכו את ה*LeaseID* ואת מנגנון *KeepAlive* האוטומטי.
- קיום ה-*Session* מייצג את חיות המנהיג. ברגע שה-*Session* נסגר, או שפג תוקפו, הצומת מאבד את המנהיגות.
- *Election (* concurrency. Election)*: אובייקט שמהווה הפשטה (abstraction) המנהל את הdistributed lock, שהוא מנגנון לוגי המבטיח מניעה הדדית בין צמתים, כך שהצומת הראשון שיוצר את המפתח נחשב כמי שמחזיק את המנעול. מנגנון זה מבטיח שיהיה מנהיג יחיד. מעבר לכך מסופק הפונקציות *Observe*, *Campaign*. הפונקציה *Campaign* היא פונקציה חוסמת המנהלת את המרוץ למנהיגות של הצמתים. הפונקציה *Observe* הינה פונקציה עבור הצמתים העוקבים עבור סנכרון הסטטוס של הקלאסטר.
- *isLeader (bool)*: משתנה מקומי המוגן ב*sync. RWMutex*. משמש לקריאה מהירה עבור אינדיקציה אם לשרת מסוים קיים בקשות כתיבה או שלא, במידה ולא קיים לא נצטרך לעבור בתוך תקשורת *etcd* שהיא יקרה.
- *leaderID(int32)*: משתנה מקומי המוגן ב*sync. RWMutex*. שומר את המזהה של המנהיג הנוכחי, עבור ניתוב בקשות.
- *stable (bool)*: דגל עבור אופטימיזציה של Multi-Paxos שיתואר בהמשך. מסמן אם המנהיג ביצע את שלב *prepare* בהצלחה, כך שבהצעות הבאות ידלג עליו.

מנגנון זיהוי הכשלים

מנגנון הכשלים במערכת מסתמך על תכונת (TTL) Time To Live של etcd.

- יצירת Session : בתחילת תהליך הבחירות, StartCampaign, הצומת יוצר Session חדש עם TTL של 5 שניות.
- Keep Alive : בעזרת ספריית concurrency, נשלח הודעת Heartbeat אל etcd ברקע על מנת לחדש את Leases כל עוד התהליך (המנהיג) חי.
- זיהוי נפילה : אם המנהיג קורס או מתנתק מהרשת Heartbeat נפסק. לאחר 5 שניות ה-TTL etcd מסתיים וה-Session נסגר בצד השרת. הנעילה של המנהיגות משוחררת אוטומטית (ואז הולכים למנגנון בחירת מנהיג).
- ויתור על המנהיגות המקומית עבור צמתים עוקבים מתבצע בעזרת הקשבה של כל צומת אל `session.Done()`. במקרה שה-Session מסתיים, הצמתים העוקבים מוותרים על המנהיג `(setLeader(false))`.

תרחישי הכשל והטיפול בהם

השימוש במנגנון TTL וה-Session שתואר לעיל מספק מענה למנגנון הכשלים של המערכת. נסביר כיצד המנגנון שתואר מתמודד עם כל תרחיש כשל:

סוג הכשל	הדרישה	הבעיה	הפתרון בעזרת etcd	הסבר על מנגנון הטיפול בכשל
קריסת צומת	זיהוי תוך זמן חסום Bounded time	תהליך השרת מת אבד.	זיהוי דטרמיניסטי של הצמתים	במקרה ששרת מת, KeepAlive שלו נפסק, ה-TTL פוקע ב-etcd, המפתח נמחק, שאר הצמתים מקבלים DELETE דרך Watchers.
ניתוק רשת	בטיחות Safety single leader	צומת חי, אך מנותק מהרשת. במידה והיה המנהיג, עלול לחשוש שהוא עדיין המנהיג.	מניעת split brain	הצומת המנותק לא מחדש את Leases מול etcd, ה-Session מתבטל, הצמתים מזהים זאת ומשחררים את המנהיגות המקומית.
צומת איטי	בטיחות Safety zombie prevention	צומת קופא לזמן מה	הגנה מפני מנהיג קפוא	אם הקיפאון ארוך מה-TTL, Leases מסתיים, הצומת מודח מהמנהיגות. לאחר שיחזור מהקיפאון, יגלה שאיבד את הנעילה ויהפוך לעוקב.
כשל של מנהיג	התאוששות אוטומטית Automatic recovery	המערכת מאבדת יכולת לקבל החלטות לאחר כשל מנהיג – צורך קיום liveness	Failover מהיר	Etcd משחרר את הנעילה לאחר שמסתיים ה-Leases, הצומת הבא בתור בחירת המנהיגות. תופס את המנהיגות.

טבלה 1: טיפול בכשלים אל מול דרישות הפרויקט

מנגנון בחירת המנהיג

תהליך בחירת המנהיג ממומש ב-StartCampaign בלולאה אינסופית עד לבחירה של מנהיג.

שלב 1: ביצוע הקמפיין

כל הצמתים החיים מנסים לקרוא ראשונים לפונקציה `elect_manager.Election.Campaign` וליצור מפתח ייחודי. אם קיים כבר מנהיג, הפונקציה נחסמת וממתינה שהמנעול ישתחרר. במידה והצומת קראה לפונקציה ראשונה (אין מנהיג), היא מייצרת את המפתח הייחודי, נועלת את המערכת ונהיית המנהיג של הקלאסטר.

שלב 2: הכרזה וייצוב

ברגע שצומת מסוימת מקבלת את המנהיגות : המנהיג הנבחר מעדכן את המצב הפנימי של `ElectionManager` להיות `isLeader = True`, ואת `leaderID` של עצמו. מודפס לוג למערכת. המנהיג מחזיר במנהיגות בתוך לולאת `select`, עד לסיום או קריסה של ה-Session.

שלב 3: מעקב של העוקבים (follower observation)

חלק זה נועד עבור מעקב של שאר הצמתים בקלאסטר אחר מי המנהיג. תהליך זה מתנהל ב-watchLeadership בעזרת פונקציית *Election.Observe(ctx)*. פונקציית העזר מחזירה ערוץ המקבל עדכון בכל פעם שהמנהיג מתחלף ב-etcd. כאשר זה קורה, העוקב משנה את leaderID המקומי.

חשיבה ארכיטקטונית:

שימוש במנעול מבוזר אטומי בעת בחירת המנהיג, מונע את בעיית ריבוי המנהיגים בצורה דטרמיניסטית. מנגנון זה מבטיח כי רק בקשה אחת תנעל את הערך בכל רגע נתון, מה שמספק *safety* חזק למערכת.

אופטימיזציית Multi-Paxos

המחלקה ElectionManager מממשת אופטימיזציה של stable leader עבור ייעול של ה-multi-paxos.

- *MarkStable(stable bool)*: במידה ומנהיג מסיים בהצלחה את שלב prepare הראשון שלו, הוא קורא לפונקציה הזו עם *stable=true*.
- *IsStableLeader()*: לפני שליחת הצעה חדשה, פרוטוקול Paxos בודק את הדגל. אם הוא true, הפרוטוקול מדלג על שלב Prepare ועובר ישירות אל שלב Accept וחוסך round trip time יקר ברשת.
- איפוס: במקרה של איבוד מנהיגות או התחלת קמפיין חדש, הדגל מתאפס אוטומטית עבור הבטחת correctness: *elect_manager.MarkStable(false)*.

חשיבה ארכיטקטונית:

מימוש המזהה של מנהיגות יציבה נובע מכך שהזמן לביצוע *Prepare* אל מול כל הקלאסטר הינו יקר, ומיותר כאשר יש לנו מנהיג יציב. שימוש במנגנון זה מקטין את הפרוטוקול משתי שלבים אל שלב אחד בלבד עבור כל כתיבה, ומגדיל את *throughput* בצורה משמעותית. האיפוס במימוש הוא קריטי, זאת מכיוון שכאשר מנהיג מוחלף, אנחנו צריכים לבצע את שני השלבים של הפרוטוקול *Paxos*, על מנת ללמוד על ההחלטות שהתקבלו ולמנוע איבוד מידע.

Membership

לאחר הבטחת קיום מנהיג וזיהוי הכשלים, נתאר את מנגנון הנוכחות של הקלאסטר. מנגנון זה ממומש ב-membership.go, במחלקה MembershipManager. במחלקה זו מתואר תמונת המצב העדכנית, ה-View, של העמיתים, Peers, של הצומת. רשימה זו היא קריטית עבור חישובי הקוורום וניתוב הודעות בפרוטוקול Paxos. מעבר לניהול הבסיסי של הרשימה, ה-MembershipManager מספק תשתית דינמית מבוססת אירועים. כלומר, התשתית מגיבה לשינויי המערכת, הצטרפות או עזיבת שרתים בזמן אמת, ללא תקשורת מיותרת של תשאול ובכך חוסכת משאבי רשת.

מבנה הנתונים של MembershipManager:

- *EtdClient*: חיבור לקלאסטר etcd. דרך משתנה זה מתבצע רישום והאזנה.
- *Peers (map[int32]string)*: מפה המוגנת על ידי *sync.RWMutex*. המפה מחזיקה את המיפוי בין האינדקס של השרת *ServerID* לבין כתובת IP שלו. המפה מייצגת את הקיום של כל שרת ברשת.
- *Callback*: פונקציה המאפשרת ל-MembershipManager לעדכן את PaxosServer בכל שינוי ברשת. במידה וצומת מצטרפת או עוזבת, השרת יכול לפתוח או לסגור חיבור gRPC בצורה מיידית.

מנגנון הפעולה

המערכת מנצלת את יכולת Watch של etcd ומחולקת לשלבים הבאים:

שלב 1 – רישום שרתים וחיות (Registration and KeepAlive)

מתבצע בעזרת הפונקציה StartMembership בצורה הבאה:

עבור כל שרת שנוסף נוצר Lease עם TTL של 10 שניות. לאחר מכן, Keep Alive מופעל ברקע יחד עם Leases שנקבע כל עוד הצומת חי. הצומת רושם את הIP שלו תחת המפתח nodes/ServerID/ ומחבר אותו אל Leases. כך שקיום המפתח מעיד על חיות הצומת. אם הצומת קורס Leases מסתיים והמפתח נמחק אוטומטית בעזרת etcd.

שלב 2 – האזנה לשינויים (Watching Peers)

מתבצע בעזרת הפונקציה watchPeers, אשר אחראית על הסנכרון בשתי שלבים:

- בטעינה הראשונית: בעת עליה של צומת, הצומת מבצע Get לכל Prefix של nodes, כדי לבנות תמונת מצב התחלתית.
- Watch loop: הצומת פותח ערוץ Watch על Prefix. כל שינוי של מצב השרתים בקלאסטר, יתקבל כאירוע WatchResponse בזמן אמת.

שלב 3 – עדכונים

מתבצע בעזרת הפונקציה handleUpdate, אשר מאבדת את האירועים במערכת ומעדכנת את המצב המקומי:

- PUT – אירוע הצטרפות של שרת: כאשר צומת חדש עולה, המערכת מצרפת אותו למפת Peers ומפעילה Callback עם `isJoin = true`, אשר נועד עבור פתיחה של חיבור gRPC חדש.
- DELETE – אירוע מחיקה של שרת לאחר עזיבה/קריסה: לאחר שLease של צומת מסתיים והמפתח נמחק, המערכת מסירה אותו ממפת Peers ומפעילה Callback עם `isJoin = false`, אשר נועדה לסיים את חיבור הgRPC.

חשיבה ארכיטקטונית:

הmembership נותן גמישות למערכת. בעזרת מימוש זה, מימוש *Adaptive Quorum* שתיארנו בפרק הקודם מתבצע תמיד מול המספר האמיתי של הצמתים החיים במערכת. כך מנגנון *quorum* מתאים את עצמו מיד למערכת האמיתי. הניהול הדינמי עוזר לנו למנוע קיפאון של המערכת ומבטיח שהמערכת תתפקד תחת כשלים משמעותיים. יתרון נוסף הוא שהזיהוי הדינמי של הצמתים חוסך עומס רשת, ע"י חיסכון בצורך של כל צומת בתשאול אם שאר הצמתים בחיים.

Consensus Messages for Multiple Instances – Multi-Paxos

בפרק זה נתאר את מימוש Multi-Paxos במערכת המבוזרת. שימוש במנגנון זה מאפשר לנו לבצע הסכמה על סדרה של החלטות, log של החלטות, ולא של ערך בודד. המערכת מנהלת מופע של MultiPaxosInstance המאגד תחתיו את כלל מופעי PaxosInstance הבודדים, תוך כדי הבטחה של סדר פעולה ליניארי של פקודות.

הרחבה של מבנה הנתונים של gRPC Protobuf Messages

בפרק 1 תיארנו את ההודעות הבסיסיות שבהן PaxosInstance משתמש. נמשיך ונפרט על הודעות התקשורת ב-gRPC בין השרתים. נחלק את ההודעות לשני סוגים: הודעות הליבה עבור instance יחיד אשר תוארו בפרק 1 והודעות עזר. נזכיר כי הודעות הליבה של המערכת הן: PrepareRequest, PromiseResponse, AcceptRequest, AcceptedResponse, CommitRequest.

הודעות העזר הנוספות:

- **ProposeRequest**: מטרת הודעה זו היא לקיים את דרישת המערכת, שלקוח יכול לשלוח בקשה לכל שרת ולקבל תשובה. כך שבמצב שבו לקוח שולח בקשת כתיבה לשרת שהוא עוקב ולא מנהיג, נשתמש בהודעה כדי להעביר את הבקשה למנהיג. כך שלאחר הצעת הבקשה הפונקציה SubmitRequest מזהה כי הבקשה לא התקבלה מהמנהיג וקוראת לפונקציית ForwardPropose ב-gRPC.
מבנה ההודעה:
 - **Value (bytes)**: תוכן הבקשה המקורית, או Batch הבקשות של חלקו.
 - **SenderId (int32)**: מזהה הצומת המעביר את ההודעה.
- שתי ההודעות הבאות נועדו על מנת להבטיח קריאות ליניאריות:
- **ReadIndexRequest**: הודעה ריקה המהווה שאילתה למנהיג, עבור בקשת האינדקס העדכני ביותר בקלאסטר.
 - **ReadIndexResponse**: תשובת המנהיג לשאילתה ReadIndexRequest. אשר מכילה את האינדקס הגבוה ביותר שהמנהיג יודע ש-Committed ע"י קוורום ונכתב ללוג. העוקב יחכה עד שמכונת המצבים שלו תגיע לאינדקס המתקבל מפונקציה זו, לפני שעונה ללקוח.
 - מבנה ההודעה:
 - **LeaderCommitIndex (int32)**: האינדקס העדכני ביותר אצל המנהיג עבורו בוצע Commit.

מבנה הנתונים MultiPaxosInstance

מחלקה זו מנהלת את PaxosInstance היחידים ומנהלת את לוגיקת Multi Paxos:

- **Instances (map[int32]*PaxosInstance)**: מבנה הנתונים המרכזי, מחזיק את כל ההיסטוריה של הלוג. שימוש במפה מאפשר לנו גמישות בזיכרון, וגישה מהירה לכל אינדקס.
- **NextIndex (int32)**: אינדקס המתאר את האינדקס הפנוי הבא במפה. המנהיג משתמש במשתנה זה על מנת להקצות מספרים סידוריים לבקשות חדשות בסדר כרונולוגי.
- **NumServers (int)**: משתנה המייצג את כמות השרתים בקלאסטר בכל זמן נתון, מתעדכן דינמית ע"י MembershipManager בעת הצטרפות או עזיבה של צמתים, ומשומש לחישוב Quorum.
- **Rows, Cols (int)**: משתנים המגדירים את הממדים של המטריצה הלוגית של הקלאסטר. משומשת ע"י החלטות grid quorum, כאשר יש במערכת שלנו יותר מ-10 שרתים.

מבנה הנתונים PaxosServer

המחלקה שמחברת בין לוגיקת Multi Paxos אל העולם החיצוני: אפליקציה, דיסק, רשת. היא מכילה את השדות הבאים:

- requestChan: ערוץ המשמש כ-buffer לקליטת בקשות מהלקוחות. משתנה זה משמש את מנגנון batching לאיגוד בקשות לפני שליחתן לקונצנוס.
- WAL (*storage.WALManager): ממשק אל Write Ahead Log שנתאר בהמשך. מבטיח כי כל שינוי ב-MultiInstances ישמר לדיסק קשיח בצורה עקבית לפני שליחת תשובה ברשת, עבור שמירה על השרידות של המערכת.
- LastApplied (int32): משתנה counter, אשר ערכו מתאר את האינדקס של הפקודה האחרונה שבוצעה במכונת המצבים AuctionSM. משתנה זה הוא אינדיקטור להבדל בין מה שהוסכם לבצע Committed, לבין מה שבוצע Applies בפועל, וע"י כך מוודא את ביצוע הפעולות הליניאריות.

פרוטוקול Multi-Paxos

פרוטוקול Multi Paxos מתבצע כפי שתיארנו בפרק הראשון. הפעם במקום לבצע את הפרוטוקול על הודעה יחידה. אנחנו מבצעים את הפרוטוקול על Batch של הודעות על MultiInstances. נראה כי כפי שתיארנו בפרקים הקודמים, כאשר מצב המערכת יהיה יציב אנחנו נדלג על השלב הראשון של Prepare עבור הגדלת Throughput.

נראה כי עבור MultiPaxosInstance נוסף מספר מתודות:

- אתחול עצל: הפונקציה GetInstance מממשת אתחול עצל. עבור הודעת רשת שמתקבלת עבור אינדקס עתידי שטרם נוצר, המערכת יוצרת אותו דינמית באותו רגע. מנגנון הזה גורם לכך שניתן יהיה שצמתים ישתתפו בקונצנוס על אינדקסים עתידיים, גם אם הם בפיגור רגעי של אינדקסים קודמים.
- חישוב quorum תלוי מערכת: נראה כי פונקציות ה-quorum ע"י HasWriteQuorum, HasElectionQuorum, מקבלות את הפרמטר MultiPaxosInstance. מה שמראה כי הלוגיקה של Instance תלוי בשינוי של הקלאסטר הכללי במערכת, מבלי לשנות את המידע עצמו, אם הוא PaxosInstance יחיד או MultiPaxosInstance.
- נראה כי ה-Instances תומכים במחיקה סלקטיבית של אינדקסים ישנים. נפרט על כך יותר בהמשך.

חשיבה ארכיטקטונית:

בחירה ב-Map דינמי עבור MultiPaxosInstance יחד עם אתחול עצל, במקום בשימוש במערך רציף, עוזר להתמודד עם out of order packets, עם עיבוד בקשות עתידיות לפני הגעה של בקשות קודמות. מנגנון זה עוסק למנוע את התקיעות של המערכת. המערכת קולטת מידע עתידי ושומרת אותו, עד השלמת הפערים, מה שמגדיל את היציבות של המערכת ועושה אותה יותר רובסטית.

ניהול בקשות לקוח בעזרת MultiPaxos

תהליך טיפול של בקשת לקוח ב-Multi Paxos מורכבת מארבעה שלהם עיקריים הממומשים ב-PaxosServer, אשר מבטיח שכל בקשה תגיע למנהיג, תגיע לקונצנוס ותבוצע בצורה סדרתית.

שלב 1: קליטה וניתוב של בקשה

לקוח שולח בקשה למערכת שלנו (יכול להתקבל ע"י כל שרת בקלאסטר), אשר מתקבלת בפונקציה SubmitRequest. אם השרת הוא המנהיג: הבקשה נכנסת לrequestChan מיד שהוא העיבוד המקומי של הבקשות. אם השרת הוא עוקב: הפונקציה SubmitRequest מזהה שהשרת אינו המנהיג, היא מאתרת את ה-ID של המנהיג הנוכחי ושולחת את הבקשה אליו באמצעות ה-ForwardPropose RPC. מנגנון זה נועד לייצר שקיפות ללקוח, כך שיוכל לעבוד מול כתובת יחידה בכל מצב.

שלב 2: איגוד בקשות (batching optimization)

על מנת לשפר את throughput של המערכת ולהקטין את עומס הרשת, המנהיג מפעיל Paxos על batch של בקשות, על MultiPaxosInstance. הפונקציה processBatches מאזינה אל requestChan ואוספת בקשות אל תוך Buffer.

Batch הבקשות נשלח כאשר אחד מהתנאים מתקיים:

1. הצטברו 100 בקשות (batchSize=100)
2. עבור 200 מילישניות מאז שליחת הבקשה (batchTimeout)

לאחר שליחת batch הבקשות, הפונקציה flushBatch ממירה את אוסף הבקשות לבלוק בינארי יחיד json array ושולחת אותו לקונצנזוס כערך אחד – Value.

חשיבה ארכיטקטונית:

שימוש בbatching קריטי הן לרוחב הפס של הרשת והן להפחתת עומס בדיסק. עבור המערכת הנוכחית, אנחנו כותבים את המידע לWAL, כפי שנתאר בהמשך. ללא batching, כתיבה לדיסק הייתה צריכה להתבצע אחד אחד, ודורשת מספר סבבי רשת כמספר הבקשות. איגוד הבקשות, הופך את העלות של הכתיבה לנמוכה הרבה יותר ומאפשרת לתמוך באלפי בקשות לשנייה.

שלב 3: הקצאה וקונצנזוס

המנהיג מקצה לBatch אינדקס ייחודי ע"י פונקציית NextInstance(), ומפעילה את פרוטוקול Multi Paxos. בשלב זה הבקשה משוכפלת לדיסק (WAL) אצל quorum הצמתים.

שלב 4: ביצוע סדרתי

לאחר שמנגנון multi paxos הסתיים והאינדקס של Batch הבקשות סומן כ Committed, נעביר אותו לביצוע של האפליקציה:

- השרת מחלץ את הBatch, ע"י פירוק הjson ומזין את הפקודות אחת אחרי השנייה למכונת המצבים.
- ברקע מבוצע לולאה של applyLog בכל השרתים, אשר עוקבת אחר המונה LastApplied.
- ישנה הבטחה על סדר הבקשות, לא נבצע פעולה באפליקציה לפני שביצענו את כל קודמותיה, מה שמבטיח ליניארזיביליות.

מקביליות הודעות – Pipeline

על מנת לנצל את רוחב הפס של הרשת, המערכת משתמשת בPipeline, שמאפשר בטיפול של מספר בקשות במקביל. בmulti paxos רגיל, המנהיג מחכה לcommit של אינדקס i לפני שמטפל בבקשה ה-i+1. מה שגורם לניצול נמוך של רוחב הפס של הרשת. לשם כך אנו משתמשים במונה pipelineLimit. המנהיג שולח מספר בקשות לביצוע – מספר Propose, הכוללים אינדקסים עתידיים, עד ל-200 בקשות פתוחות. נשתמש ב-Instances שהיא המפה של MultiPaxosInstance, עבור יצירת אינדקסים לא רציפים בזיכרון. כך שברקע פונקציית applyLog מיישרת את הביצועים בתוך Multi Paxos לפני שנשלח לאפליקציה.

חשיבה ארכיטקטונית:

בעזרת מנגנון זה אנחנו יכולים להפריד רשויות (decoupling) בין שכבת התקשורת (שכבת הקונצנזוס) לשכבת האפליקציה. כך ששכבת התקשורת עובדת בצורה מקבילית ואסינכרונית, על מנת לנצל את הרשת. בעוד ששכבת האפליקציה עובדת בצורה סדרתית, על מנת להבטיח את הנכונות של המערכת (correctness). כך בעצם אנחנו מנצלים את היתרונות של מערכת מקבילית וגם שומרים על המידע קונסיסטנטי.

קריאות ליניארזביליות

המערכת מאפשר ביצוע של קריאות ליניארזביליות, שמאפשרות קריאה בטוחה מכל צומת של הערך העדכני ביותר. המנגנון ממומש ע"י HandleRead בצורה הבאה:

1. בקשת הקריאה: השרת העוקב מקבל בקשת קריאה ושולח הודעת ReadIndexRequest למנהיג.
2. תגובת המנהיג: המנהיג מחזיר את LeaderCommitIndex, האינדקס העדכני ביותר שידוע לו ש-Committed.
3. העוקב משווה את האינדקס שקיבל את LastApplied שלו. כך שאם האינדקס שלו לא עדכני מספיק הוא יחכה ע"י applyCond.Wait() עד שהערך שלו יתעדכן.
4. לאחר שמכונת המצבים של השרת התעדכנה, העוקב מחזיר את התשובה של הקריאה אל הלקוח. כך שמובטח שהערך שהחזיר ללקוח הינה העדכנית ביותר.

חשיבה ארכיטקטונית:

במקום להעביר כל קריאה דרך מנגנון הקונצנזוס ולייצר צוואר בקבוק של I/O ועומס גדול על הדיסק ועל הלוג, אנחנו משתמשים בordering עבור total ordering, שהינו מנגנון קל מבחינת גודל המידע שנשלח. מנגנון זה דורש סבב יחיד על השרת של המנהיג על מנת לקבל את האינדקס העדכני ביותר, וחוסך גישה אל הדיסק. כך בעצם אנחנו מבטיחים עקביות ושמירה על ביצועים מהירים.

Replicated Log and State Machine

שכבת האפליקציה במערכת המבוזרת, הינה שכבה הבנויה מעל מנוע הקונצנזוס של ה multi paxos שתיארנו בפרקים הקודמים. שכבה זו היא מימוש של Replicated State Machine, מכונת המצבים המשוכפלת. בשכבה זו אנו לוקחים את log הפקודות שהתקבל מה multi paxos וממירים אותו למצב עדכני במערכת שלנו, וחשיפתו ללקוח. עבור מערכת המצבים שאנחנו בחרנו לממש, היא מערכת מצבים של מערכת מכרזים מבוזרת.

מנגנוני התקשורת במערכת

נראה כי באופן כללי מנגנון paxos אחראי על ניתוב הבקשות בצורה עקבית, ומשתמש בשתי ממשקי תקשורת, גם כאשר הלקוח פונה לשרת שאינו המנהיג. כפי שתיארנו, מנגנון paxos בעל שני ממשקי הקשבה של תקשורת. כך שכל שרת PaxosServer מקבל הודעות בשתי ממשקים במקביל:

- RESTful API – שרת HTTP חיצוני המאזין בפורט 80xx. חלק זה של השרת משמש לקליטת בקשות לקוח (כמו bid, create) ולהחזרת הודעות בפורמט json. מנגנון זה עוזר להסתיר מהלקוח את מורכבות ה cluster מאחורי הקלעים.
- gRPC פנימי – שרת RPC בינארי, המאזין בפורט נפרד. משמש לתקשורת מהירה ויעילה בין הצמתים ב cluster. השימוש ב protobuf מבטיח סריאליזציה יעילה ומהירה של הודעות פנימיות של המנגנון עצמו (כמו Prepare, Accept, Commit).

מכונת המצבים – Auction State Machine

מכונת המצבים מנהלת את המצב העסקי של המערכת. היא מקבלת פעולות סדרתיות – replicated logs מה multi paxos ומעדכנת את המצב הפנימי של המערכת בצורה דטרמיניסטית. המימוש על ה state machine מתבצע במחלקה AuctionStateMachine.

מבני הנתונים הבסיסיים של המערכת

AuctionItem - מכרז

מכרז הינו אובייקט המתאר מכרז בודד במערכת. מורכב מ:

- ItemID (int32) : מזהה ייחודי לפריט (ובכך גם למכרז)
- ItemName (string) : שם הפריט\המכרז
- HighestBid (float64) : סכום ההצעה המובילה במכרז כרגע. מתעדכן בצורה מונוטונית עולה.
- WinnerID (int32) : המזהה של המשתמש שנתן את ההצעה הגבוהה ביותר עד כה.
- CreatorID (int32) : המזהה של המשתמש שפתח את המכרז. משמש לאכיפת הרשאות.
- IsOpen (bool) : דגל המייצג את סטטוס המכרז. אם true המכרז פעיל לקבל הצעות, אם false המכרז נסגר.

מבנה סוג הפקודות - CommandType

מבנה המכיל את סוג הפקודה שניגש אליה על פי ערך הstring שקיבלנו – ENUM של ערכי הפקודה:

- PlaceBid
- CloseAuction
- CreateAuction
- DeleteAuction
- RegisterUser

מבנה הפקודה - Command

מבנה המשוכפל לכל השרתים במערכת, המייצג את המבנה שנכתב ללוג של הפקסוס ומפעיל את השינויים באפליקציה של כל בקשה. מורכב מ:

- Type (CommandType) : סוג הפעולה לביצוע.
- UserID (int32) : המשתמש שיזם את הפעולה. משמש עבור אימות מול מפת הUsers.
- ItemID (int32) : המכרז עליו מתבצעת הפעולה.
- Amount (float64) : סכום הצעה על פריט (רלוונטי רק ל PlaceBid)
- Timestamp (int64) : חותמת זמן ייחודית עבור הבקשה (בנו שניות)

חשיבה ארכיטקטונית:

Timestamp משמש כמזהה ייחודי של הבקשה, ומשומש לצורך התאמה בין הבקשה שנשלחה לבין התשובה שהתקבלה, ומאפשר סנכרון בין הHTTP אל Paxos.

מבנה הנתונים של ה- State Machine

כל מצב state מוחזק בזיכרון RAM בצורה הבאה:

- Items (map[int32]*AuctionItem) : מפה הממפה בין מזהים - ID של מכרזים לבין האובייקט המלא של מצב המכרז – staten המלא.
- Users (map[int32]bool) : רשימת המשתמשים הרשומים במערכת. מעזרת רשימה זו אנחנו אוכפים הרשאות. לדוגמא, רק משתמש רשום יכול ליצור מכרז.
- NextItemID (int32) : מונה המייצג את ה-ID הבא שיוקצה למכרז שייוצר, מוקצה בצורה סדרתית ועקבית.
- NextUserID (int32) : מונה המייצג את ה-ID הבא שיוקצה למשתמש שנרשם למערכת, מוקצה בצורה סדרתית ועוקבת.
- Responses (map[int64]chan) : מפת ערוצים עבור ניהול המצב המערכתי. המפה מגשרת בין Multi-Paxos הא-סינכרוני לבין הHTTP הסינכרוני. המפתח הוא Timestamp של הפקודה, והערך הוא הערוץ עליו ממתין הthread שמטפל בלקוח.

- `LastAppliesIdx (int64)` : משתנה המכיל את האינדקס האחרון שהמכונה ביצעה. שדה זה משמש לטיפול בקריאות ליניאריות, ע"י אינדיקציה של סנכרון כל שרת עם המנהיג.

עיבוד בקשה במכונת המצבים

תהליך הבקשה במכונת המצבים מחולק אל שני שלבים, על מנת להגן על המערכת מעומס ולהבטיח לוגיקה נכונה :

שלב 1: אימות מקדים PreCheck

לפני שנשלחת פקודה אל מנגנון Multi Paxos שהינו משאב יקרת הכולל כתיבה לדיסק, סריאליזציה ותקשורת אל מול כל הקלאסט, היא עוברת אימות מקדים בתוך ה-PaxosServer, בעזרת AuctionStateMachine.PreCheck. שלב זה בודק את תקינות הבקשה ודוחה בקשות שגויות, כך שהדחייה נעשית מוקדם, ללא הכנסת זבל אל תוך הלוג של Multi Paxos. האימות מתבצע בצורה הבאה :

- מבוצעות בדיקות statesless validation : מבוצעות ע"י הפונקציה Basic_Valid. אלו בדיקות שאינן דורשות גישה לזיכרון המשותף.
- בדיקות stateful validation : מבוצעות ע"י validateStates תחת נעילה. זאת מיוון שבדיקות אלו מבצעות קריאה של המצב הנוכחי של המכרזים והמשתמשים.

הערה : אנחנו משתמשים ב-PreCheck כבדיקה מקדימה לאופטימיזציה של ביצועים. אך בזמן שהפקודה ממתינה לקונצנזוס ייתכן ופקודה אחרת תשנה את המצב של המערכת. לדוגמא, אם מישו הציע הצעה יותר גבוהה. לכן, בדיקה זו משמשת כאינדיקציה ראשונית והתקינות הסופית תיבדק לאחר מכן.

שלב 2: קונצנזוס וסדר במערכת

בשלב זה, הפקודה עוברת דרך מנגנון WAL, Batching, Multi Paxos, כפי שתואר בפרק 3, עד שהיא הופכת ל-Committed ונכנסת ללוג של השרתים.

שלב 3: ביצוע הפעולה במערכת (Apply)

פקודות שעברו קונצנזוס עוברות לפונקציית Apply. הפקודה בשלב הזה, הינה מוטמעת בזיכרון, והפונקציה צריכה לעדכן את מכונת המצבים בהתאם. מתבצעות הפעולות הבאות :

- נעילה של המפות של מכונת המצבים : Items, Users, Responses. זאת על מנת להבטיח כי רק פקודה אחת מעדכנת את המצב בכל רגע נתון, וע"י כך להבטיח Atomicity.
- אימות חוזר : מתבצע אימות נוסף של בדיקות stateful validation, למקרה בו והמצב השתנה מאז ביצוע ה-PreCheck. בשלב זה כל handler פנימי, על פי סוג הפקודה, קורא שוב validateStates.
- שינוי המצב באפליקציה : לאחר ביצוע האימות, המערכת מעדכנת את הזיכרון על פי סוג הפקודה (cmd.Type).
- עדכון המידע המערכתי : LastAppliedIdx מתעדכן לאינדקס של הלוג הנוכחי logIdx.

שלב 4: נויטיפיקציה ללקוח

לאחר שהשינוי בוצע, או נכשל, המערכת תודיע ללקוח שממתין ב-HTTP Response. בצורה הבאה :

המערכת בודקת אם קיים ערוץ פתוח במפה Responses תחת המפתח של הבקשה (cmd.Timestamp). אם קיים, היא שולחת את התוצאה לפי התגובה המתאימה אל הערוץ. שליחת התשובה אינה חוסמת את המערכת במידה והלקוח התנתק. לאחר מכן, הערוץ נמחק מהמפה על מנת למנוע דליפה של הזיכרון.

פקודות מכונת המצבים והלוגיקה של המערכת

כל פקודה שמגיעה למערכת, עוברת סדרת פעולות אטומיות, תחת נעילה *s. Mu. Lock*, המבטיחה עקביות של המערכת. נפרט על הלוגיקה של כל פקודה שממומשת ב-auction.goo.

- רישום משתמש – RegisterUser:
 - בעזרת פעולה זו יוצרים משתמש חדש במערכת.
 - הרישום מתבצע כך שמקצים מזהה חדש מ-NextUserID ומקדמים את המונה לאחר הוספת המשתמש למפת Users.
 - המשתמש מקבל ID ייחודי המאפשר לו לבצע פעולות כתיבה במערכת (יצירת מכרז, הגשת הצעה, סגירת מכרז של עצמו, מחיקת מכרז של עצמו).
- יצירת מכרז – CreateAuction:
 - בעזרת פעולה זו, משתמש יוצר מכרז במערכת.
 - תנאים נדרשים: המשתמש חייב להיות רשום במערכת.
 - יצירת המכרז מתבצעת ע"י הקצאת ItemID חדשת יצירת אובייקט AuctionItem עם סטטוס HighestBid, IsOpen=true ראשוני והוספה למפת Items.
- הגשת הצעת מחיר במכרז – PlaceBid:
 - פעולה עבור הצעה של משתמש על מכרז מסוים.
 - תנאים נדרשים:
 - האם המכרז קיים ופתוח – IsOpen=true
 - האם ההצעה ה-bid הינו מונוטוני גבוה יותר מההצעה הנוכחית.
 - האם מי שמציע את המחיר אינו היוצר. נועד למניעה של הרצת מחיר עצמית
 - האם המשתמש שמציע את הbid אינו המשתמש המוביל כרגע: $WinnerID \neq UserID$. נועד למניעת הצעות כפולות של המשתמש המוביל.
 - מתבצע ע"י עדכון השדות HighestBid, WinnerID של אובייקט המכרז לערכים החדשים.
- סגירת מכרז – CloseAuction:
 - סגירת מכרז קיים של משתמש מסוים על ידיו.
 - תנאים נדרשים: רק יוצר המכרז (ע"י זיהוי CreatorID) רשאי לסגור את המכרז.
 - פעולה בלתי הפיכה, לאחר סגירת המכרז, לא ניתן להגיש יותר הצעות על המכרז ונקבע המנצח של המכרז.
- מחיקת מכרז – DeleteAuction:
 - מחיקת מכרז קיים ע"י היוצר שלו.
 - תנאים נדרשים: רק היוצר יכול למחוק את המכרז.
 - מחיקת המפתח של המכרז ממפת Items, ע"י ה-ID שלו בתוך המפה.

Node Recovery and Synchronization

במערכת מבוססת, קריסת שרתים הינה אירוע הנדרש לטיפול המערכת. על מנת להבטיח את העמידות (durability) והזמינות של המערכת. המערכת חייבת להשתמש במנגנוני שמירה של נתוני המערכת על מנת לסנכרן שרת חדש שחזר לאחר נפילה, או שהצטרף, במהירות ויעילות, ללא איבוד מידע. בפרק זה נתאר את פרוטוקול ההתאוששות של שרת למצב פעיל ומסונכרן. המימוש ב-server.go מבדיל בין שני תרחישים: שחזור מקומי לאחר קריסה וסנכרון רשת לאחר פיגור מתמשך. בפרק זה רק נתאר את תהליך ההתאוששות והסנכרון ונסביר על שמירת הנתונים בהמשך.

הרחבה של מבנה הנתונים של gRPC Protobuf Messages

בפרק 1 ופרק 3 תיארנו את ההודעות הבסיסיות שבהן PaxosInstance משתמש. נמשיך ונפרט על הודעות התקשורת בgRPC בין השרתים שנועדו לתמוך בתהליך הסנכרון.

הודעות הנתונים עבור סנכרון שרת מוגדרות בקבצי הproto וכוללות את השדות:

- **GetLogRequest**: הודעה המכילה רק את ID של השרת שמבקש את המידע. השרת המקבל מזהה את המבקש ומחליט איזה מידע לשלוח לו.
 - **sender_id**: מזהה של השרת ששולח את הבקשה.
- **GetLogResponse**: הודעה תשובה משרת מעודכן עם המידע לסנכרון:
 - **next_index (int32)**: האינדקס הפנוי הבא אצל השרת המשיב. נותן אינדיקציה על אורך הלוג שנמצא בשרת המעודכן.
 - **log_entries (repeated LogEntry)**: רשימת הפקודות החסרות. כל **LogEntry** מכיל **instance_id** ואת ה**value** (הפקודה).
 - **snapshot_index (int32)**: האינדקס הגבוה ביותר שנשמר ב**snapshot**

תרחיש 1: שחזור מקומי

בעת עליית שרת ב**Start**, בפעולה הראשונה של השרת היא ניסיון שחזור של המצב מהדיסק המקומי לפני יצירת קשר עם השרת. תהליך זה מתבצע ע"י:

- טעינת **snapshots**: השרת בודק האם קיים קובץ **snapshots**. אם כן, טוען אותו למכונת המצבים ומעדכן את **LastApplied** לאינדקס המתקבל מ**snapshot**.
- טעינת **WAL**: השרת קורא את לוג הפקודות מהדיסק. פעולות בעלות **LastApplied** אינדקס גדול יותר מהאינדקס של ה**snapshot** נטען לזיכרון של השרת ב-**MultiInstance**.

תרחיש 2: סנכרון רשת - SyncLog

סנכרון השרת עם הרשת נובע ממגוון סיבות, הכוללת ביניהן את **node recovery**, אך אינה ייחודית עבור נפילת חומרה. פעילות הסנכרון של שרת מבוצע ע"י פונקציית **SyncLog**. סיבות אפשריות לסנכרון של שרת הינן:

- קיטוע רשת: השרת היה מבודד מהקלאסטר באופן זמני עקב תקלת תקשורת. בזמן הניתוק קלאסטר המשיך לקבל החלטות בלעדי השרת. לאחר חידוש החיבור, השרת נדרש להסתנכרן.
- איבוד הודעות: עומס רשת מנע מהודעת **commit** מסוימת להגיע לשרת ויצרה חור בלוג של השרת, שנדרש להשלים ע"י סנכרון.
- עומס ואיטיות רשת: השרת חווה איטיות רגעית ופספס סבב קונצנזוס.
- הצטרפות שרת חדש ו**node recovery**: לאחר השחזור המקומי בתרחיש 1, השרת שעלה עדיין יכול להיות בפיגור אחרי הקלאסטר ונדרש סנכרון.

תהליך הסנכרון:

- **discovery** – תשאול: השרת שולח בקשת **GetLogRequest** לכל העמיתים **Peers** בקלאסטר.
- בחירת המקור לסנכרון: השרת אוסף את התשובות ובוחר את הצומת המסונכרנת ביותר ע"פ התנאים הבאים:
 - הצומת מחזיק את ה**SnapshotIndex** הגבוה ביותר
 - במקרה של שוויון, מי מחזיק בלוג בעדכני ביותר – ה**NextIndex** הגבוה ביותר.
- סנכרון השרת: השרת משווה את המצב שלו אל המצב של העמית הנבחר ומחליט על אסטרטגיית סנכרון:

- השלמת מידע מהירה - Fast Forward : אם הפער קטן והלוגים המעודכנים קיימים אצל העמית, השרת מוריד את הפקודות החסרות, מריץ אותן על עצמו ובכך מסנכרן את עצמו.
- דריסה – InstallSnapshot : אם פער המידע גדול מאוד ולוגים ישנים כבר נמחקו אצל העמית. השרת מבקש את הSnapshot המלא, דורס את המצב המקומי ובכך מסנכרן את עצמו.

חשיבה ארכיטקטונית:

מנגנון הrecovery והסנכרון הזה מאפשר למערכת להיות אלסטית. שרת חדש לחלוטין יכול להצטרף למערכת ואינו צריך ללמוד את כל ההיסטוריה של המערכת, תהליך שהינו יקר ואיטי. השרת רק צריך להעתיק את תמונת המצב העדכני ביותר של המערכת ויכול להמשיך משם.

Snapshots and Consensus Disposal

במערכת מבוצעת עם replicated log, היסטוריית הפקודות גדלה באופן ליניארי עם הזמן. ללא מנגנון תחזוקה, השמירה של כל ההיסטוריה בזיכרון RAM ובדיסק יוביל למספר בעיות עיקריות:

1. קריסת משאבים: הזיכרון יתמלא לחלוטין, out of memory, והשרת יקרוס.
2. זמן התאוששות ארוך: בעת אתחול שרת מחדשת שרת יצטרך לעדכן מיליוני פקודות מהלוג על מנת להגיע למצב העדכני, תהליך שייקח זמן רב.

על מנת לפתור את הבעיות הללו המערכת משתמשת בsnapshots. תמונות המצב ממירות רצף ארוך של log אל מצב סופי דחוס של state יחיד, הנשמר בקובץ יחיד. בעזרת מנגנון זה, נוכל להריץ את המערכת לאורך זמן תוך שמירה קבועה של משאבים.

הרחבה של מבנה הנתונים של gRPC Protobuf Messages

בפרקים הקודמים תיארנו את ההודעות הבסיסיות שבהן המערכת משתמשת. נמשיך ונפרט על הודעות התקשורת בgRPC בין השרתים שנועדו לתמוך בתהליך ההטמעה של snapshot.

כאשר נרצה לעדכן את השרת ע"י הורדת snapshot נשתמש בהודעות הנתונים מוגדרות בקבצי הproto שכוללות את השדות:

- InstallSnapshotRequest : מכיל את המידע המלא של הsnapshot העדכני מהעמית.
 - sender_id (int32) : השרת השולח את הsnapshot
 - term (int32) : הTerm הנוכחי, עבור אימות מנהיגות.
 - last_included_index (int32) : האינדקס האחרון שהsnapshot בשרת השולח מכסה.
 - data (bytes) : הקובץ הבינארי של הsnapshot עצמו.
- InstallSnapshotResponse : אישור קבלה ואישור עדכון בהצלחה.
 - term (int32) : הTerm הנוכחי של השרת המקבל. משמש לעדכון האינדקס של המנהיג.
 - success (bool) : משתנה בינארי אם עודכן בהצלחה או לא.
- StatusResponse : הודעה המשמשת לניטור המערכת עבור סנכרון השרתים ובעיר עבור מנגנון מחיקת הלוגים.
 - appliedIndex (int32) : האינדקס האחרון שהשרת המשיב ביצע (Apply) במכונת המצבים. שדה זה משמש את המנהיג לחישוב המינימום הגלובלי של ביצוע פעולה עבור הקלאסטר לפני מחיקת הלוגים.
 - isLeader (bool) : דגל המציין האם השרת המשיב רואה את עצמו כמנהיג. לצורך דיבוג ובקרה.

מבנה הנתונים של SnapshotData

מבנה הנתונים snapshot השומר את תמונת המצב של האפליקציה (AuctionStateMachine) עבור שמירה שלו בקובץ. לאחר שמירה של המידע במבנה זה, המידע עובר סריאליזציה לjson. המבנה של SnapshotData נמצא ב-auction.go ומכיל את השדות הבאים:

- Items (map[int32]*AuctionItem): העתק של המפה של בסיס הנתונים. (הממפה בין מזהים - ID של מכרזים לבין האובייקט המלא של מצב המכרז – staten המלא).
- Users (map[int32]bool): העתק של מפת רשימת המשתמשים הרשומים במערכת.
- NextID (int32): המצב העדכני של מונה המייצג את ה-ID הבא שיוקצה למכרז שייווצר, מוקצה בצורה סדרתית ועקבית.
- NextUserID (int32): המצב העדכני של המונה המייצג את ה-ID הבא שיוקצה למשתמש שנרשם למערכת, מוקצה בצורה סדרתית ועוקבת.
- LastIndex (int32): משתנה המכיל את האינדקס האחרון שהמכונה ביצעה שמוכלל ב-snapshot. שדה זה משמש כנקודת תיאום בסנכרון של השרת אל מול ה-WAL כפי שתואר בתרחיש 1 בפרק הקודם. כך ששרת שעובר אתחול מקומי יידע לטעון מהלוג רק את הפקודות שהגיעו לאחר אינדקס זה.

יצירה ושמירה של Snapshot

יצירת תמונת המצב עבור המערכת מתבצע בצורה תקופתית. נרצה ליצור snapshot בצורה תכופה ולשמור לדיסק על מנת לא לאבד מידע. מצד שני כתיבה תכופה לדיסק הינה פעולה איטית עלולה ליצור צוואר בקבוק ב-CPU. לשם כך ביצענו הפרדה בין יצירת snapshot אל השמירה שלו.

יצירה של תמונת המצב

היצירה מבוצעת לאחר דרישה בפונקציית applyLog, הנמצאת ב-server.go.

פונקציה זו רצה ב-PaxosServer כלולאה אינסופית המקבלת פקודות שאושרו (committed) ומחילה אותן על מכונת המצב. לאחר החלת batch של פקודות במכונת המצבים וקבלת האינדיקציה של ההחלה שלהן, נשמור את תמונת המצב הנוכחית. יצירת תמונת המצב מתבצעת בתוך פונקציית go func applyLog ע"י קריאה ל-s.AuctionSM.SaveSnapshot(s.SnapshotPath, idx). נראה כי הפונקציה go func הינה פונקציה א-סינכרונית. applyLog לא מחכה שהשמירה לדיסק תסתיים. אלא ממשיך לטפל בפקודות הבאות. זאת על מנת למנוע חסימה של הפרוטוקול עקב איטיות של השמירה לדיסק. אנחנו מעבירים לפונקציה האסינכרונית את האינדקס s.LastApplied הנוכחי, כערך שיצביע שמירת snapshot על נקודת הזמן שנדרש לשמור, גם אם המערכת התקדמה אל מצבים חדשים.

שמירה אטומית של תמונת המצב

שמירה של תמונת המצב מתבצעת ע"י פונקציית SaveSnapshot ב-auction.go. בעת שמירת snapshot הפונקציה מוודאה כי הקובץ ישמור תמונת מצב אמיתי ולא מושחת, גם אם שרת קרס במהלך הכתיבה. השמירה מתבצעת במספר שלבים:

שלב 1: נעילה וסריאליזציה

השלב היחיד שבו המכונה ננעלת. הפונקציה json.Marshal קוראת את המפות SnapshotData בשלב זה אנחנו מבצעים נעילה על מנת שלא יתבצע קריסה והשחתה של הקובץ על ידי כתיבה מעל המידע הנוכחי. נראה כי אנחנו מבצעים אופטימיזציה לנעילה. הנעילה משתחררת מיד לאחר Marshal ולפני הכתיבה לדיסק. כלומר הנעילה מתבצעת רק לזמן הקצר של העתקת הנתונים לbuffer. כך שהכתיבה האיטית לדיסק מתבצעת ללא נעילה, מה שמאפשר לשרת להמשיך לקבל הודעות במקביל לכתיבה.

שלב 2: כתיבה לקובץ זמני

הכתיבה נעשית לקובץ זמני tmp. כך שבמקרה של כשל במהלך הכתיבה לא נשמור קובץ מושחת בדיסק. הקובץ האמיתי snapshot.json לא נפגע וכשנעלה מחדש, נטען את נקובץ התקין הישן והמערכת תתעלם מהקובץ הזמני המיכל מידע מושחת – הוא לעולם לא יהיה במערכת.

שלב 3: החלפה אטומית

הפקודה os.Rename היא פקודה אטומית אשר משנה את המצביע בטבלת הקבצים אל המשתנה החדש. המצביע משנה את ערכו לקובץ הזמני ברגע שסיים לכתוב ונעשה הקובץ הקבוע במערכת של snapshot. ברגע שפעולה זו הצליחה ה-snapshot נחשב כתקין ושלים.

גיוזום הלוגים – Safe Log Compaction

יצירת snapshot מפנה מקום בדיסק אך לא משחרר את הזיכרון RAM של השרת. נשתמש בתהליך ייעודי לניקוי הלוגים. המחיקה של הלוגים תריכה להיות זהירה מאוד על מנת לא לדרוס מידע ששרתים אחרים בקלאסטר לא קיבלו. לכן על מנת לנקות את הלוגים נבצע זאת רק לאחר קבלת מצב גלובלי של הקלאסטר. גם במקרה הזה, הניקוי של הלוגים מתבצע במספר שלבים.

תזמון הניקוי ואחריות

תהליך הניקוי דורש תשאול של כל השרתים ברשת ולכן הינו פעולה יקרה שלא נרצה לבצע עבור כל בקשה יחידה. האחריות לתת את ההוראה לנקות את הלוגים היא של המנהיג בלבד. העוקבים מנקים את הלוגים שלהם רק לאחר הוראה מהמנהיג, או בעת קבלת snapshot. נתזמן את ניקוי הלוגים בתוך פונקציית go func בתוך Startn. בפונקציה זו מוגדר Ticker שרץ אחד ל-60 שניות עבור ניקוי הלוגים. תדירות זו מהווה trade off בין ניקוי הזיכרון אל החיסכון בתעבורת רשת.

הערה ארכיטקטונית: נראה כי השרת שלנו מוגן מפני פיצוץ זיכרון – out of memory בשישים שניות. בעזרת שני מנגנוני הגנה: השילוב של batching שמקטין את כמות האובייקטים, ובעזרת מנגנון pipeline limiting שעוצר את הכניסה כשהמערכת עמוסה.

ניקוי הלוגים

ניקוי הלוגים מתבצע ע"י פונקציית DisposeOldLogs ב-server.go. הניקוי מתבצע במספר שלבים:

שלב 1: איסוף המידע הגלובלי – GetMinApplied.

המנהיג צריך לוודא מהו המכנה המשותף הנמוך ביותר בקלאסטר שבוצע במערכת המצבים. המנהיג קורא לפונקציית GetMinApplied אשר יוצרת רשימה של כל Peers המחוברים, שולחת בקשת GetServerStatus RPC לכולם. השרתים משיבים עם האינדקס של הפעולה האחרונה שביצעו, AppliedIndex שלהם. המנהיג לוקח את האינדקס הנמוך ביותר מבין האינדקסים שהתקבלו. ערך זה מייצג את נקודת הזמן שבה מובטח שכל השרתים בקלאסטר מחזיקים במידע.

שלב 2: חישוב סף המחיקה

נראה כי גם אם כל השרתים הגיעו לאינדקס מסוים, לא נרצה למחוק בדיוק עד ערך זה. זאת מכיוון שרעידות ברשת או איטיות רגעית עלולות לגרום לשרת לבקש לוג שהרגע נמחק. לכן נשתמש בbuffer=50. כך שנמחק 50 לוגים פחות מהערך שקיבלנו. ה buffer משמש אזור ביניים שבו שרתים איטיים יוכלו לבצע השלמת פערים מהירה בעזרת הפקודות שנשמרו בbuffer במקום להוריד קובץ snapshot שלם, שהינו תהליך יקר מבחינת רוחב פס וזמן עיבוד המידע.

שלב 3: מחיקה של הלוגים

לאחר חישוב הthreshold, השרת מבצע את המחיקה בפועל: המערכת נועלת את הגישה לזיכרון. השרת עובר על מפת Instances ומוחקת את המופעים ב-RAM אשר האינדקסים שלהם נמוכים מהסף שקבענו. השרת מדפיס ללוג כמה רשומות פונו לצורך בקרה וניטור. נראה כי תהליך זה שקוף ללקוח ומתבצע אוטומטית ע"י המנהיג ללא התערבות ידנית.

מנגנון הורדת תמונת המצב למערכת – InstallSnapshot

תיארנו את השימוש במנגנון זה בפרק הקודם וכעת נצלול ונתאר כיצד עובד המנגנון עצמו. כפי שתיארנו בפרק הקודם, המנגנון מופעל כאשר יש פער גדול בין שרת מעוקב אל המנהיג. מצב זה קורה כאשר הלוגים שהשרת צריך נמחקו מהקלאסטר, ובפרט המנהיג. במקרה זה המנהיג חייב להתקין על השרת המעוקב את תמונת המצב המלאה והעדכנית ביותר. נראה כי כאשר השרת מתקין את snapshot אנחנו מבצעים נעילה של השרת לפעולות אחרות, מכיוון שאנו מחליפים את הזיכרון של השרת, והשינוי הזה צריך להיות שינוי אטומי, אנו חייבים לנעול את השרת עבור כל התהליך הזה.

טיפול הבקשה – InstallSnapshot Handler

הפונקציה InstallSnapshot בserver.go מטפלת בבקשת RPC ומבצעת דרישה מבוקרת של המצב המקומי של השרת המעוקב. בשלבים הבאים:

שלב 1: בדיקת רלוונטיות

לפני ביצוע פעולה של כתיבה לדיסק והתקנה של snapshot, השרת בודק האם snapshot שהתקבל אכן חדש יותר ממצבו הנוכחי. הוא עושה זאת ע"י בדיקה של האינדקס האחרון שמומש בsnapshot אל מול האינדקס האחרון שבוצע במכונת המצבים אצלו. במידה והאינדקס אינו עדכני יותר, השרת מתעלם מהבקשה.

שלב 2: כתיבה לדיסק

השרת כותב את המידע הבינארי req.Data, שהתקבל מהשרת ישירות לקובץ snapshot מקומי ב-s.snapshotPath. כך שגם במידה והשרת יקרוס לאחר מכן, המידע העדכני יהיה זמין לו במהלך ההתאוששות.

שלב 3: טעינה למכונת המצבים

השרת קורא לפונקציה s.AuctionSM.LoadSnapshot ב-auction.go. פעולה זו מפרמטת את json וטוענת את המידע של מכונת המצבים לזיכרון ה-RAM של השרת, תוך כדי דריסת המצב הקודם.

שלב 4: עדכון אינדקסים

השרת מעדכן את מצביעי paxos שלו לערכים העדכניים שהתקין. הוא מעדכן את s.LastApplied שלו לאינדקס של snapshot. הוא מעדכן את s.MultiInstance.NextIndex אל LastApplied+1.

שלב 5: ניקוי לוגים אגרסיבי

לאחר שההיסטוריה מותקנת בשרת על גבי snapshot, השרת מוחק את הלוגים הישנים בזיכרון, וע"י כך השרת מתיישר עם המצב של שאר הקלאסטר.

Data Storage - WAL

כחלק מהשמירה לזיכרון, עבור מערכת מבוזרת, נרצה לשמור מידע לא רק בזיכרון ה-RAM אלא גם בדיסק קשיח. זאת על מנת להבטיח עקביות ועמידות של המערכת מפני קריסות שרתים. בעזרת מנגנון Write Ahead Log שמימשנו wal.go, נבטיח שהמצב בדיסק יישמר.

מבנה הנתונים של WALManager

ה-WALManager הינו רכיב עצמאי שמנהל את הכתיבה לקובץ בדיסק. הוא מכיל:

- Path: נתיב הקובץ בדיסק

- `lastData (ByteBuffer)` : משתנה המחזיק את הנתונים העדכניים בפורמט json שעבור המרה לבתים. מייצג את תמונת המצב האחרונה שנשמרה בזיכרון RAM, לפני או במהלך תהליך הכתיבה לדיסק. גם בזמן ההמתנה ל-`counter` של שינויים, השרת עובד מול הנתונים העדכניים ביותר ב-RAM, גם אם לא נכתבו עדיין לדיסק.
- `counter (int)` : מונה המשמש ליישום batching עבור שיפור ביצועים. המונה סופר את כמות עדכוני המצב מאז השמירה האחרונה. עבור מינימום של 10 שינויים נבצע את השמירה הקשיחה לדיסק (`syncToDisk`). זה חוסך גישות יקרות לדיסק עבור כל עדכון קטן.

שמירה לדיסק – PersistState

הפונקציה `PersistState` הינה הפונקציה המרכזית ב-WAL. אשר מתבצעת בשלבים הבאים :

שלב 1: Deep copy

הפונקציה יוצרת העתק מלא של כלל המופעים, ה-`instances`. מתבצע מעבר על כל ה-`instances` והעתקה של כל המפות הפנימיות למפות חדשות כשאר אנחנו נוועלים את הגישה לזיכרון בשלב הזה בלבד. אנחנו מבצעים את השלב הזה, בגלל שברקע `paxos` ממשיך לרוץ ולשנות את הזיכרון. לכן אם היינו מפרמטים את המידע בזמן שהוא משתנה היינו מקבלים מידע לא עקבי או שגיאות `race conditions`. לאחר שלב זה אנחנו משחררים את הזיכרון ובכך מבצעים אופטימיזציה של `latency`.

שלב 2: json marshaling

המידע מומר לפורמט json, שהינו פורמט נוח עבור דיבוג.

שלב 3: batching - השהייה

נכתוב אל הדיסק רק כאשר ה-`counter` - כמות השינויים עובר את הסף הנדרש, במקרה שלנו 10 שינויים. שלב זה מקפיץ את הביצועים של המערכת, מכיוון ששמירה לדיסק הינה יקרה בזיכרון.

שלב 4: atomic write

הפונקציה `syncToDisk` מבצעת את הכתיבה לדיסק בצורה בטוחה : אנחנו כותבים את המידע לקובץ זמני ואז מבצעים `os.Rename` כדי להחליף את הקובץ הישן בחדש. כתיבה זו מבטיחה כי לא יקרה מצב של קובץ מושחת או לא שלם שנשמר בדיסק.

אינטגרציה של WAL עם PaxosServer

השרת משתמש ב-WAL בשתי נקודות זמן עיקריות : בזמן ריצה (כתיבה של פעולה) ובזמן עלייה (קריאה).

כתיבה : בכל שלב בפרוטוקול Multi-Paxos, השרת שומר את המצב בדיסק בצורה אסינכרונית. לאחר הבטחת `promise` של שרת, לאחר קבלת ערך `accept` ולאחר `commit`. אנחנו משתמשים בפונקציית `gon` האסינכרונית, כך שהשרת לא מחכה שהשרת יסיים לכתוב לדיסק לפני שחוזר אל הלקוח. בכך אנחנו משפרים את הביצועים `latency`, אך מחלישים את העמידות במקרה של נפילות חשמל. נראה כי אנחנו מתגברים על החיסרון הזה בדרכים אחרות : שימוש ב-`quorum` עבור אישור של פעולה. כך שגם עם שורת ספציפי איבד את המידע האחרון, המידע קיים אצל שרתים אחרים במערכת שקיבלו אותו כבר. יתר על כן מנגנון `SyncLog` מסנכרן את השרת שלנו לאחר פונקציית `Start`, לאחר הקריאה מה-WAL. כפי שתואר בפרק ה-`Node Recovery and Synchronization`.

קריאה : בזמן עלייה של שרת וביצוע `recovery`. הפונקציה `Start` מופעלת. השרת קורא אל `Wal.LoadState()`. השרת טוען את כל המופעים `instances` מהדיסק לזיכרון. השרת מחשב את `NextIndex` של סמך המידע שנטען.

Client API

Client API אשר מוגדר ב-auction_api.go הינו הרכיב האחראי על תרגום בקשות ה RESTful API (HTTP) אל מול שאר פקודות המערכת.

ניהול הinterface (HTTP Server)

הפונקציה StartAuctionInterface מגדירה את השרת :

- Routing : שימוש ב-http.NewServeMux לניתוב נתיבים (paths).
- CORS Wrapper : ה-API עוטר את הבקשות בעזרת middleware המאפשר גישה מדפדפנים, ומאפשר חיבור front end חיצוני.
- Dynamic Port : כל שרת פותח פורט HTTP ייחודי המבוסס על ה-ID שלו.

בקשות הכתיבה Linearizability Write

בקשות הכתיבה הינן בקשות ליניארזיביליות. הן מתבצעות ב-HandleAuctionRequest. בקשות אלו מתבצעות בשלבים הבאים :

1. Json decoding : Handler מפרסר את Body של ה-POST לתוך מבנה ה-Command.
2. Pre-consensus validation : ה-API קורא אל AuctionSM.PreCheck. כפי שתיארנו בפרק 3.
3. Response Channel : ה-API יוצר ערוץ make(chan string,1) ושומר אותו בזיכרון השרת. ערוץ זה הוא הקשר בין HTTP thread שנדרש להחזיר תשובה ללקוח אל consensus thread שמבצע את הפעולה.
4. Submission and wait : ה-API שולח את הבקשה אל Paxos וממתין ל-select. כך שאם שתתקבל תשובה מהערוץ, או שיעבור 30 שניות (timeout) והלקוח יקבל הודעת שגיאה.

בקשות הקריאה הליניארזיביליות

בקשות הקריאה הליניארזיביליות משתמשות במנגנון WaitUntilSynced על מנת להבטיח קריאה עדכנית מהמערכת. כפי שתיארנו בפרק 3, השרת במקומי עוצר את בקשת ה-HTTP, פונה למנהיג כגדי לדעת מה האינדקס האחרון שאושר וממתין שהלוג המקומי יתעדכן לאותו אינדקס. לאחר מכן ה-API יחזיר תשובה ללקוח.

בקשות הקריאה הסדרתיות

Handler ניגש ישירות אל ה-state machine המקומי ושולף את המידע ישירות. ה-API ממיר את המפות והמערכים הפנימיים אל json ומשדר אותם ללקוח.

Batching API

נקודת הקצה batch/ מאפשרת אופטימיזציה של המערכת. הלקוח שולח רשימה של פקודות json array. ה-API מקצה לכל פקודה מפתח Timestamp ומקצה Response Channel משלו בתוך מפה פנימית במכונת המצבים. המפה המקומית נשלחת כחבילה יחידה אל מנגנון Multi-Pxos. ה-API ממתין בתוך לולאה עד שכל בקשות ה-Batch מסתיימות, מאגד את הבקשות למערך ומחזיר ללקוח מערך של תוצאות.

RESTful API Endpoints

Linearizable Write Endpoints

נתיב (Path)	שיטה	תיאור	דוגמת קלט	דוגמת פלט
register/	POST	רישום משתמש חדש וקבלת UserID ייחודי.	{ }	{ "status": "Success", "user_id": 1 }
create/	POST	יצירת מכירה פומבית חדשה עבור פריט מסוים.	{ "item_name": "Camera", "amount": 1200, "user_id": 1 }	{ "status": "Success", "item_id": 101 }
bid/	POST	הגשת הצעת מחיר (חייבת להיות גבוהה מההצעה הנוכחית).	{ "item_id": 101, "user_id": 2, "amount": 1300 }	{ "status": "Success", "message": "Success: Bid accepted" }
close/	POST	סגירת המכירה (מתבצע רק על ידי יוצר המכירה).	{ "item_id": 101, "user_id": 1 }	{ "status": "Success", "message": "Success: Auction closed" }
delete/	POST	מחיקה מוחלטת של מכירה מהמערכת.	{ "item_id": 101, "user_id": 1 }	{ "status": "Success", "message": "Success: Auction deleted" }
batch/	POST	שליחת מערך של פקודות לביצוע מרוכז ואופטימלי.	{ "type": "PLACE_BID", "item_id": 101, "user_id": 3, "amount": 1400 }, ... }	{ "status": "Success", "batch_results": ["Success: Bid accepted"] }

Linearizable Reading Endpoints

נתיב (Path)	שיטה	תיאור	דוגמת שאילתה	דוגמת פלט
status/bid/sync/	GET	קבלת הצעה אחרונה וזוכה נוכחי בצורה מסונכרנת.	?id=101	{ "item_id": 101, "winner": 3, "amount": 1400 }
status/total/sync/	GET	חישוב סך ההכנסות הגלובליות במערכת (מכירות סגורות).	{ }	{ "total_system_value": 5400.5 }
status/creator/total/sync/	GET	חישוב סך ההכנסות עבור יוצר ספציפי לאחר סנכרון מלא.	?user_id=1	{ "user_id": 1, "total_value": 2500, "item_count": 2 }

Sequential Reading Endpoints

נתיב (Path)	שיטה	תיאור	דוגמת שאילתה	דוגמת פלט
status/	GET	קבלת רשימת כל הפריטים (מכירות) במערכת.	{ }	{ { "item_name": "Camera", "item_id": 101, ... }, ... }
active/	GET	סינון והצגת מכירות פומביות פתוחות בלבד.	{ }	{ { "item_name": "Phone", "is_open": true, ... } }
status/name/	GET	חיפוש פריטים לפי שם (name : Query Param).	?Camera=1	{ { "item_name": "Camera", "item_id": 101, ... } }
status/item_id/	GET	שליפת מידע על פריט ספציפי לפי מזהה (item_id).	?item_id=101	{ "item_name": "Camera", "item_id": 101, "highest_bid": 1400, ... }
status/user_id/	GET	הצגת כל המכירות שנוצרו על ידי משתמש מסוים (user_id).	?user_id=1	{ "user_id": 1, " "auctions_created": [...] }

Sequential System Reading Endpoints

נתיב (Path)	שיטה	תיאור
info/	GET	מציג את מזהה השרת, ה-Applied Index והאם הוא Leader.
debug/peers/	GET	מציג סטטוס רשת: כמות Peers, Quorum Type וקונפיגורציה.
system/logs/	GET	ערוץ SSE (Server-Sent Events) לצפייה בלוגים של הקונצנוס.

Docker

ה-docker נועד לענות על הדרישה של הפצת המערכת בסביבה מבוססת בקנה מידה רחב, תוך שימוש בקוטיינרים לכל שרת, עבור הפרדת הרכיבים השונים. המערכת נפרסת בעזרת Docker Compose, המאפשר להרים את כל הקלאסטר (שרתי paxos, שרתי etcd, הgateway) בפקודה אחת.

תהליך הבנייה Docker file

ה-dockern מורכב מmulti stage build על מנת ליצור image סופי קל ומהיר. שלב הבנייה (builder): שימוש בimages רשמי של golang: 1.25-alpine. בשלב זה מועתקים קבצי go.mod וsum, מתבצעת הורדות תלויות והקוד מקומפל לקובץ בינארי יחיד בשם paxos_server.

שלב הריצה (runtime): שימוש ב-alpine: 3.19 כבסיס קטן ומאובטח. הקובץ הבינארי שוכן מועתק לשלב זה, מה שמקטין את נפח הimage הסופי.

ארכיטקטורת הפריסה Docker Compose

קובץ ה-docker-compose.yaml מגדיר קלאסטר מורכב הכולל את הרכיבים הבאים:

- Etcd: רכיב התשתית הראשון שעולה. משמש לניהול חברות בקבוצה (membership), זיהוי כשלים ובחירת מנהיג. השרתים מוגדרים עם depends_on ו-healthcheck מול etcd כדי לוודא שלא יתחילו לפעול לפני שתשתית התיאום מוכנה.
- Paxos Servers (0-11): הקלאסטר הכולל 12 שרתים עצמאיים, כל שרת בנוי על פני קוטיינר עצמאי, המריצים את המערכת על כל קונטיינר. כל שרת מקבל מזהה יחודי ומתחבר ל-etcd.
- Networking: כל הרכיבים מחוברים לרשת פנימית מבודדת מסוג bridge בשם paxos-net.

שער הכניסה – NGINX Gateway

על מנת לפשט את התקשורת עבור הלקוח, הוספת שרת NGINX שעובד כreserve proxy ו-load balancer.

- שער הכניסה מאפשר נקודת כניסה אחודה. שער הכניסה הופך את הבקשה של הלקוח אל המערכת לשקופה, הלקוח לא צריך להכיר את כתובות השרתים, הוא פונה לפרט יחיד של gateway.
- Upstream cluster: ה-NGINX מנהל את הקבוצה paxos_cluster המכילה את כל השרתים (server0-server11).
- תמיכה ב-CORS: nginx.conf כוללת טיפול בבקשות OPTIONS, והוספת Headers של Access Control, המאפשר לפתח ממשק משתמש frontend בדפדפן, שיכול לדבר עם API ללא חסימות אבטחה.

Testing and Client Demo

בפרק זה נבצע מבחנים למערכת המבוססת שהצגנו, נציג את העמידה בדרישות המערכת (specifications) ונציג את ה-demo של בקשות לקוח וקבלת תגובה מהמערכת.

תהליך הבנייה וקמפול המערכת

נציג חלק מההדפסות, להוכחת הבנייה :

```
[+] up 27/27
✓Image docker-server1      Built      206.1s
✓Image docker-server2      Built      206.1s
✓Image docker-server7      Built      206.1s
✓Image docker-server11     Built      206.1s
✓Image docker-server9      Built      206.1s
✓Image docker-server6      Built      206.1s
✓Image docker-server4      Built      206.1s
✓Image docker-server0      Built      206.1s
✓Image docker-server3      Built      206.1s
✓Image docker-server8      Built      206.1s
✓Image docker-server10     Built      206.1s
✓Image docker-server5      Built      206.1s
✓Network docker-paxos-net   Created     0.0s
✓Container docker-etcd-1    Created     0.2s
✓Container docker-server6-1 Created     0.2s
✓Container docker-server2-1 Created     0.4s
✓Container docker-server8-1 Created     0.4s
✓Container docker-server10-1 Created     0.4s
✓Container docker-server7-1 Created     0.3s
✓Container docker-server5-1 Created     0.3s
✓Container docker-server1-1 Created     0.4s
✓Container docker-server0-1 Created     0.4s
✓Container docker-server4-1 Created     0.4s
✓Container docker-server9-1 Created     0.2s
✓Container docker-server3-1 Created     0.4s
✓Container docker-server11-1 Created     0.4s
✓Container docker-gateway-1 Created     0.1s
Attaching to etcd-1, gateway-1, server0-1, server1-1, server10-1, server11-1, server2-1, server3-1, server4-1, server5-1,
server6-1, server7-1, server8-1, server9-1
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
Container docker-etcd-1    Waiting
```

```
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy w Confi
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy
Container docker-etcd-1    Healthy
```

נוכל לראות בתמונה הראשונה כי רכיבי המערכת סיימו להיבנות build-12. השרתים, שרת gateway, וצומת etcd נבנו בהצלחה, הקונטיינרים נוצרו והופלו בתוך רשת paxos-net. זמן הבנייה של המערכת: 206 שניות. נוכל לראות בתמונה השנייה את לוג ההרצה של etcd המשמש coordination services עבור בחירת leader ב-paxos. הלוגים מאשרים כי הקונטיינרים במצב בריא ואפשר להמשיך לתהליך paxos, העלאת השרתים ובחירת המנהיג.

בחירת המנהיג הראשוני.

נראה כי נבחר מנהיג ראשוני בעזרת etcd עליו הצמתים מסכימים :

```
server1-1 | 2026/02/05 09:27:34 [Server 1] starting paxos on port 50051
server1-1 | 2026/02/05 09:27:34 [Server 1] Auction REST API starting on port 8081
server1-1 | 2026/02/05 09:27:34 paxos server 1 started on 50051
server1-1 | 2026/02/05 09:27:34 [DEBUG] server 1 is try to start campaign now
server1-1 | 2026/02/05 09:27:34 [Server 1] Waiting 6s before starting SyncLog to avoid boot storm
server1-1 | 2026/02/05 09:27:34 [Election 1] Connecting to Etcd Session...

server9-1 | 2026/02/05 09:27:34 [Server 9] starting paxos on port 50059

server1-1 | 2026/02/05 09:27:34 [Election 1] Session created successfully

server1-1 | 2026/02/05 09:27:34 [Election 1] is the leader

server9-1 | 2026/02/05 09:27:34 [Election 9] Observed new leader: 1
```

נראה כי שרת 1 עלה למערכת, תפס את ההנהגה, ושרתים אחרים – כמו שרת 9 זיהו זאת מיד.

הוכחת Membership

```
server1-1 | 2026/02/05 09:27:35 [Membership] registered as 1 with IP server1:50051
```

```
server1-1 | 2026/02/05 09:27:35 [Membership] node 9 joined at server9:50059
```

נראה כי הצמתים משתמשים במנגנון membership דינמי. כל שרת שעולה יודע להכריז על עצמו ולמצוא את הכתובת שלו בתוך הdocker.

עדכון טופולוגיית ה-Dynamic Quorum

```
server1-1 | 2026/02/05 09:27:35 [Server 1] peer 9 joined. grid updated to 1x2
```

```
server1-1 | 2026/02/05 09:27:35 [Server 1] peer 8 joined. grid updated to 2x2
```

```
server10-1 | 2026/02/05 09:27:36 [Server 10] peer 5 joined. grid updated to 2x3
server9-1 | 2026/02/05 09:27:36 [Server 9] peer 10 joined. grid updated to 3x3
server1-1 | 2026/02/05 09:27:36 [Server 1] peer 0 joined. grid updated to 2x4
server4-1 | 2026/02/05 09:27:36 [Server 4] peer 0 joined. grid updated to 2x43
server6-1 | 2026/02/05 09:27:36 [Server 6] peer 11 joined. grid updated to 2x3
server0-1 | 2026/02/05 09:27:36 [Server 0] peer 9 joined. grid updated to 2x4
```

```
server9-1 | 2026/02/05 09:27:36 [Server 9] peer 7 joined. grid updated to 3x4
```

נראה כי המערכת מנהלת קלאסטר דינמי ומחשבת את מטריצת RxC על פי הצמתים שעלו למערכת בצורה דינמית ושומרת על עקביות.

התאוששות וסנכרון לוגים

נוכיח כי המנגנון ממלא את החורים בלוג ושרתים חדשים מקבלים את המצב העדכני

```
server0-1 | 2026/02/05 09:27:40 [SyncLog] starting state transfer from peers
server0-1 | 2026/02/05 09:27:40 [SyncLog] found new snapshot from peer 3, in Idx: 0
server0-1 | 2026/02/05 09:27:40 [Server 0] State saved index -1
server0-1 | 2026/02/05 09:27:40 [SyncLog] completed recovery
```

נראה כי שרת 0 עלה בהצלחה, מצא מידע שחסר לו, משך מידע מהsnapshot מצומת אחר (3) וסיים את השחזור בהצלחה.

Client Demo

נבצע כעת מספר מבחנים, על מנת להראות client demo והוכחת המערכת.

מבחן 1 – מבחן שפיות בסיסי

מטרת המבחן הינה לרשום 2 משתמשים. המשתמש הראשון ייצור מכרז, המשתמש השני ינסה לקרוא את המכרז הספציפי.

מצד הלקוח:

```
[TEST] Testing Registration, Auction Creation and Reading
[PASS] registered creator id: 3
[PASS] registered bidder id: 4
[PASS] created item id: 2
[PASS] replication verified, server http://localhost:8091 sees 'TestCouch'
```

מצד השרתים:

```
server1-1 | 2026/02/05 10:21:30 [Leader 1] Received forwarded proposal from Server 0
server1-1 | 2026/02/05 10:21:30 [Batching] Flushing batch of 1 requests
server1-1 | 2026/02/05 10:21:30 [Leader 1] skipping Prepare for instance 5 using ProposalID 11
server1-1 | 2026/02/05 10:21:30 [DEBUG 1] Phase 2 finished. Acceptors: [7 5 4 9 2 6]. Quorum Reached: true
server1-1 | 2026/02/05 10:21:30 Value committed: [{"type":"CREATE_AUCTION","item_name":"TestCouch","item_id":0,"user_id":3,"amount":100,"timestamp":1770286890636164936}]
server1-1 | 2026/02/05 10:21:30 [Leader 1] commit value=[{"type":"CREATE_AUCTION","item_name":"TestCouch","item_id":0,"user_id":3,"amount":100,"timestamp":1770286890636164936}] at index=5
server1-1 | 2026/02/05 10:21:30 [SM SUCCESS] Added Item 2 (TestCouch). Total Items: 2
server1-1 | 2026/02/05 10:21:30 [Server 1] State saved index 5
```

נוכל לראות כי 2 הלקוחות נוצרים בהצלחה ומקבלים id מהמערכת, נראה כי ה-auction נוצר בהצלחה ומקבל גם הוא id ייחודי. נראה כי מהקריאה של המכרז נראה כי השרת שפנינו אליו רואה אותו ויכול לקרוא ממנו.

נוכל לראות מצד שרתים, כי ברגע שמתבצעת הבקשה, מועבר למנהיג forward של הבקשה, 200 מילישניות עוברות ומתבצע batching של הבקשה היחידה, המנהיג יציב, לכן מדלגים על prepare. מסתיים השלב השני ויש quorum של שרתים שמסכימים ולבסוף הערך commit בקלאסטר מבוצע ב-state machine.

מבחן 2 – Batching Request

נראה כי המבחן עבר בהצלחה:

```
[TEST] Testing Linearizable Batch Requests
[PASS] Registered user for batching: 1
[PASS] Batch processed 3 commands successfully
```

מצד שרת:

```
server9-1 | 2026/02/05 12:17:31 [Leader 9] Received forwarded proposal from Server 0
```

אם נסתכל על המערכת נראה כי המנהיג קיבל את הבקשה. נשתמש ב-UI שבנינו עבור ריכוז ההודעות ונראה כי:

```
20:56:45 [SVL_00] [QUORUM] Phase 2 Success: Acceptors [1 6 3 7 5 4 0 9 10 11 2] reached quorum for Instance 1QUORUM
20:56:44 [SVL_00] [QUORUM] Phase 2 Success: Acceptors [5 6 1 3 4 7 10] reached quorum for Instance 0QUORUM
20:56:44 [SVL_00] [QUORUM] Phase 1 Success: Responders [5 11 9 3 6 10 4 7 2 0] reached quorum for Instance 0QUORUM
```

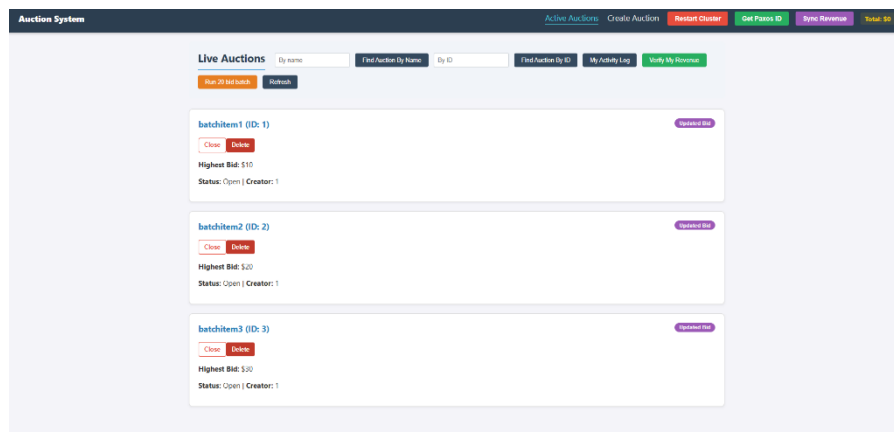
נראה כי הושג קוורום של promise ואז accept עבור ה-batch של ההודעות.

```
20:56:47 [SVL_00] [REQUEST] Follower 0 forwarded request to Leader 8
20:56:47 [SVL_00] [REQUEST] Heavy Load Detected: Received Batch of 3 commandsBATCH
20:56:47 [SVL_00] [REQUEST] New User registration initiated
20:56:47 [SVL_00] [REQUEST] Follower 0 forwarded request to Leader 8
20:56:45 [SVL_00] [REQUEST] Batching Engine: Flushing 1 accumulated requests to Paxos InstanceBATCH
20:56:44 [SVL_00] [REQUEST] Leader received 358 bytes for batchingBATCH
20:56:44 [SVL_00] [REQUEST] Batching Engine: Flushing 1 accumulated requests to Paxos InstanceBATCH
20:56:44 [SVL_00] [REQUEST] Leader received 101 bytes for batchingBATCH
```

נוכל לראות את התהליך שעבר batch ההודעות. נראה כי הוא עבר אל המנהיג, הבקשה אובדה במנגנון paxos ולאחר מכן, לאחר ביצוע ה-commit חזרה אל המנהיג.

```
20:56:54 [SVL_07] [CONSENSUS] Follower 7 committed Instance 1 from Leader 8BATCH
20:56:54 [SVL_07] [CONSENSUS] Index 1 applied to local State MachineBATCH
20:56:54 [SVL_07] [CONSENSUS] Applying batch of 3 commands to Auction SM at Index 1BATCH
20:56:53 [SVL_07] [SYSTEM] [Server 7] Accept from 8 proposal=18 accepted=true
20:56:53 [SVL_07] [CONSENSUS] Follower 7 committed Instance 0 from Leader 8BATCH
20:56:53 [SVL_07] [CONSENSUS] Index 0 applied to local State MachineBATCH
20:56:53 [SVL_07] [CONSENSUS] Applying batch of 1 commands to Auction SM at Index 0BATCH
20:56:53 [SVL_07] [SYSTEM] [Server 7] Accept from 8 proposal=18 accepted=true
20:56:53 [SVL_07] [SYSTEM] [Server 7] Prepare from 8 proposal=18 promised=true
20:56:53 [SVL_01] [MODE] Server 1 | Applied Index: 1 | Role: Follower
20:56:53 [SVL_01] [MODE] Server 1 | Applied Index: 1 | Role: Follower
20:56:53 [SVL_01] [MODE] Server 1 | Applied Index: 1 | Role: Follower
20:56:53 [SVL_01] [MODE] Server 1 | Applied Index: 1 | Role: Follower
```

נראה כי לאחר הגעה לקונצנזוס, המערכת ביצעה commit עבור מופע batching וביצעה אותו במכונת המצבים.



נוכל לראות את הבקשות קיימות כעת באפליקציה שלנו, לאחר שבוצעו.

מבחן 3 – עקביות

```
[TEST] Testing linearizable read
[PASS] linearizable read (/sync) returned correct bid: 500, it took 0.023s
```

לאחר ביצוע מבחן 1 – נבצע את מבחן 3 – ונרצה לראות אם אנחנו יכולים לקרוא קריאה ליניארזיבילית. נראה כי המבחן עבר בהצלחה. קיבלנו הודעה חזרה מהמערכת של הקריאה האמיתית.

מבחן 4 – נפילת מנהיג ו recovery

על מנת להריץ את המבחן, נריץ גם את מבחן 1. נראה כי :

```
[TEST]
Testing leader failure and failover
leader is server 8 (http://localhost:8088)
killing the leader, killing server 8
time="2026-02-05T21:52:04+02:00" level=warning msg="C:\\Users\\Liraz Calif\\
Desktop\\תורזומ תונרעמ\\סיסרוק\\project\\src\\docker\\docker-compose.yaml:
the attribute 'version' is obsolete, it will be ignored, please remove it to
avoid potential confusion"
Container docker-server8-1 Killing
Container docker-server8-1 Killed
sending write request to alive server: 9
[PASS] system recovered. Write accepted after leader failure
resurrecting server 8
time="2026-02-05T21:52:11+02:00" level=warning msg="C:\\Users\\Liraz Calif\\
Desktop\\תורזומ תונרעמ\\סיסרוק\\project\\src\\docker\\docker-compose.yaml:
the attribute 'version' is obsolete, it will be ignored, please remove it to
avoid potential confusion"
Container docker-etcd-1 Waiting
Container docker-etcd-1 Healthy
Container docker-server8-1 Starting
Container docker-server8-1 Started
[PASS] Server 8 restarted.
```

נוכל לראות כי המבחן עבר בהצלחה, הרגנו את המנהיג, הצלחנו לשלוח בקשת כתיבה שאושרה ואתחנו את המנהיג מחדש. נסתכל בצד השרת על מנת לראות את מנגנון election :

```

server3-1 | 2026/02/05 19:52:09 [Election 3] Observed new leader: 4
server11-1 | 2026/02/05 19:52:09 [Election 11] Observed new leader: 4
server4-1 | 2026/02/05 19:52:09 [Election 4] is the leader
server9-1 | 2026/02/05 19:52:09 [Election 9] Observed new leader: 4
server10-1 | 2026/02/05 19:52:09 [Election 10] Observed new leader: 4
server1-1 | 2026/02/05 19:52:09 [Election 1] Observed new leader: 4
server2-1 | 2026/02/05 19:52:09 [Election 2] Observed new leader: 4

server6-1 | 2026/02/05 19:52:09 [Election 6] Observed new leader: 4
server5-1 | 2026/02/05 19:52:09 [Election 5] Observed new leader: 4

server4-1 | 2026/02/05 19:52:09 [Election 4] Observed new leader: 4

server4-1 | 2026/02/05 19:52:11 [Leader 4] Received forwarded proposal from server 9

```

נראה כי מנהיג 4 קיבל את המנהיגות ע"י etcd בצורה מהירה. לאחר מכן בוצע בקשת הכתיבה:

21:52:11	SVR_04	[REQUEST]	Batching Engine: Flushing 1 accumulated requests to Paxos InstanceBATCH
21:52:11	SVR_04	[REQUEST]	Leader received 120 bytes for batchingBATCH
21:52:11	SVR_09	[REQUEST]	Follower 9 forwarded request to Leader 4
21:52:11	SVR_09	[REQUEST]	Client POST /create , User: 3 , Item: SurvivalItem
21:52:11	SVR_09	[REQUEST]	Follower 9 forwarded request to Leader 8
21:52:11	SVR_09	[REQUEST]	Follower 9 forwarded request to Leader 8
21:52:09	SVR_09	[REQUEST]	Client POST /create , User: 3 , Item: SurvivalItem
21:52:07	SVR_09	[REQUEST]	Client POST /create , User: 3 , Item: SurvivalItem

נוכל לראות את התהליך המלא, שבתחילה השרתים שולחים את הבקשה למנהיג הקודם, ברגע שמזהים שנפל, ונבחר מנהיג חדש, שולחים אליו והבקשה מתבצעת אצלו.

מבחן 5 – נפילת follower, סנכרון של השרת בעליה והשמת snapshot על מנת להריץ את המבחן, נריץ גם את מבחן 1. נראה כי:

```

(base) C:\Users\Liraz Calif\Desktop\מיסרוק\תורזובמ\project\src\etc>python test.py
Starting Comprehensive Distributed Systems Test...

waiting for cluster to boot system ready, 11 nodes online
[TEST] Testing registration, auction creation and reading
[PASS] registered creator id: 13
[PASS] registered bidder id: 14
[PASS] created item id: 29
[PASS] replication verified, server http://localhost:8091 sees 'TestCouch'
[TEST]
Testing follower failure and catchup using log/snapshots
victim selected: follower server 5
time="2026-02-05T22:09:36+02:00" level=warning msg="C:\\Users\\Liraz Calif\\Desktop\\מיסרוק\
\תורזובמ\project\\src\\docker\\docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
Container docker-server5-1 Killing
Container docker-server5-1 Killed
server 5 is offline
writing data to cluster without the follower
resurrecting server 5
time="2026-02-05T22:09:37+02:00" level=warning msg="C:\\Users\\Liraz Calif\\Desktop\\מיסרוק\
\תורזובמ\project\\src\\docker\\docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
Container docker-etcd-1 Waiting
Container docker-etcd-1 Healthy
Container docker-server5-1 Starting
Container docker-server5-1 Started
waiting for sync
[PASS] recovery verified, server 5 caught up and has 'missedItem4'.

```

נראה כי אנחנו הורגים את השרת, מבצעים פעולות כתיבה ומעלים אותו שוב לאחר מכן, לאחר שמחכים שהשרת יעלה לרשת מבצעים קריאה ממנו, נראה כי השרת מעודכן במצב העדכני ביותר. אכן, הוא מסתנכר.

מבחן 6 – מבחן עומס על המערכת – 1000 בקשות במקביל

```
waiting for cluster to boot system ready, 11 nodes online
[TEST]
starting 1000 clients - load test
targeting Gateway: http://localhost/api
processed 0/1000
processed 200/1000
processed 400/1000
processed 600/1000
processed 800/1000

[RESULT] processed 1000 in 4.41s (227 req/s)
[PASS] load test passed
```

נראה כי בזמן קצר, הצלחנו להעביר 1000 הודעות למערכת ולבצע אותן.

```
22:16:00 SVR_11 [NODE] Server 11 | Applied Index: 68 | Role: Follower
22:16:00 SVR_06 [NODE] Server 6 | Applied Index: 68 | Role: Follower
22:16:00 SVR_01 [NODE] Server 1 | Applied Index: 68 | Role: Follower
22:16:00 SVR_02 [NODE] Server 2 | Applied Index: 68 | Role: Follower
22:16:00 SVR_09 [NODE] Server 9 | Applied Index: 68 | Role: Follower
22:16:00 SVR_00 [NODE] Server 0 | Applied Index: 68 | Role: Follower
22:16:00 SVR_08 [NODE] Server 8 | Applied Index: 68 | Role: Follower
22:16:00 SVR_03 [NODE] Server 3 | Applied Index: 68 | Role: Follower
22:16:00 SVR_07 [NODE] Server 7 | Applied Index: 68 | Role: Follower
22:16:00 SVR_04 [NODE] Server 4 | Applied Index: 68 | Role: LEADER
22:15:59 SVR_05 [NODE] Server 5 | Applied Index: 68 | Role: Follower
22:15:58 SVR_11 [NODE] Server 11 | Applied Index: 68 | Role: Follower
22:15:58 SVR_06 [NODE] Server 6 | Applied Index: 68 | Role: Follower
22:15:58 SVR_01 [NODE] Server 1 | Applied Index: 68 | Role: Follower
22:15:58 SVR_02 [NODE] Server 2 | Applied Index: 68 | Role: Follower
22:15:58 SVR_09 [NODE] Server 9 | Applied Index: 68 | Role: Follower
22:15:58 SVR_00 [NODE] Server 0 | Applied Index: 68 | Role: Follower
22:15:58 SVR_08 [NODE] Server 8 | Applied Index: 68 | Role: Follower
22:15:58 SVR_03 [NODE] Server 3 | Applied Index: 68 | Role: Follower
22:15:58 SVR_07 [NODE] Server 7 | Applied Index: 68 | Role: Follower
22:15:58 SVR_04 [NODE] Server 4 | Applied Index: 68 | Role: LEADER
22:15:57 SVR_05 [NODE] Server 5 | Applied Index: 68 | Role: Follower
```

נראה כי המצב התקבל במערכת, ואנחנו שומרים את ה-`index=68`. מה שהגיוני, לאור כך שאנו מבצעים batching אל ה-`logging` של הבקשות. שמוכיח את יעילות ה-`batching` במערכת.

מבחן 7 – מקסימום נפילות שרתים

במבחן זה נפיל כ-5 שרתים מתוך ה-12 ונראה אם המערכת מסוגלת להתמודד עם בקשה לאחר מכן. אנחנו הורגים את השרתים הנבחרים ברנדומליות דרך ה-`docker`.

```
Testing maximum servers failure (n/2 - 1)
cluster size: 12
max failures: 5
killing 5 servers: [9, 0, 3, 7, 6]
```

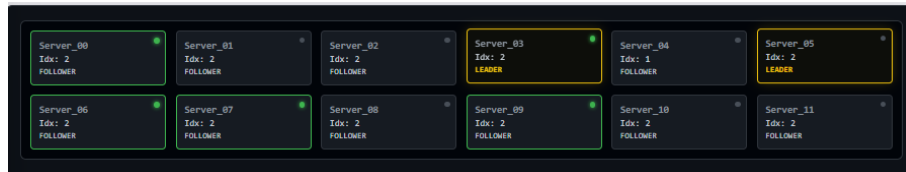
נוודא כי הם נפלו:

```
[VERIFICATION] connectivity check:
server 9: offline (confirmed: port closed)
server 0: offline (confirmed: port closed)
server 3: offline (confirmed: port closed)
server 7: offline (confirmed: port closed)
server 6: offline (confirmed: port closed)
attempting write with 5 nodes down
[PASS] system survived 5 max failures, write committed.
```

נראה כי המבחן עובר בהצלחה והמערכת מצליחה לכתוב גם בזמן של נפילה של $n/2-1$ שרתים. שרידות המערכת ומימוש ה-`membership` הדינמי יחד עם ה-`adaptive quorum` מאפשר לנו לבצע כתיבה למערכת גם לאחר כשל של יותר מ- $n/2-1$ שרתים.

עבור כשל של 7 שרתים, ו-5 שרתים חיים במערכת :

```
[VERIFICATION] connectivity check:
server 4: offline (confirmed: port closed)
server 5: offline (confirmed: port closed)
server 2: offline (confirmed: port closed)
server 8: offline (confirmed: port closed)
server 10: offline (confirmed: port closed)
server 1: offline (confirmed: port closed)
server 11: offline (confirmed: port closed)
```



מתבצע ניתוק של השרתים ואנו מאששים ניתוק זה. נראה כי בשלב זה יש במערכת 5 שרתים. נראה כי המערכת בוחרת את המנהיג הבא, לאחר שהמנהיג הנוכחי מת (הוא מסומן כמנהיג כי הוא לא פעיל ולא יודע שאינו המנהיג יותר, אך אין לו השפעה על המערכת). נראה כי מבוצעת בקשת כתיבה אשר מתקבלת ע"י 5 השרתים :

```
attempting write with 7 nodes down
[PASS] system survived 7 max failures, write committed.
```

נראה כי הכתיבה מתבצעת במערכת ויש committed לביצוע הכתיבה. נוכל לראות גם את המימוש של הפעולה במכונת המצבים :

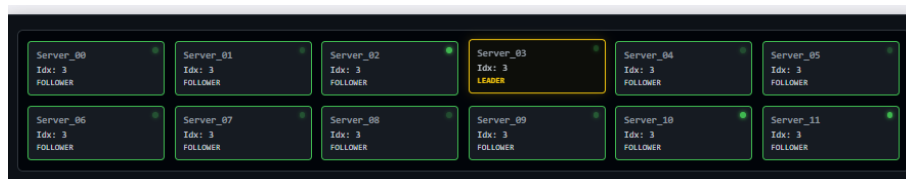
maxfailureItem (ID: 2) Updated Bid

Close Delete

Highest Bid: \$999

Status: Open | Creator: 1

לאחר העלאה מחודשת של השרתים :



המנהיג הישן רואה שאינו המנהיג יותר ומקבל את המנהיגות של שרת 3. בכך נראה כי המערכת שלנו עמידה בפני פיצול מנהיגים.

נראה כי מעבר לכך המערכת שלנו אפילו עמידה בנפילה של עד 9 שרתים :

כך שכאשר יש נפילה של 9 שרתים מנגנון quorum מתבצע ע"י majority של השרתים הנמצאים במערכת. הנפילה :

```
[VERIFICATION] connectivity check:
server 5: offline (confirmed: port closed)
server 10: offline (confirmed: port closed)
server 6: offline (confirmed: port closed)
server 1: offline (confirmed: port closed)
server 7: offline (confirmed: port closed)
server 2: offline (confirmed: port closed)
server 4: offline (confirmed: port closed)
server 9: offline (confirmed: port closed)
server 8: offline (confirmed: port closed)
```

הכתיבה :

```
attempting write with 9 nodes down  
[PASS] system survived 9 max failures, write committed.
```

המימוש באפליקציה :

maxfailureItem (ID: 4)

Updated Bid

Close

Delete

Highest Bid: \$999

Status: Open | Creator: 3

מבחן 8 – מבחן העברת הודעות מעוקב למנהיג

על מנת להריץ את המבחן, נריץ גם את מבחן 1. נראה כי :

```
[TEST]  
Testing request forwarding, from follower to leader  
leader is server 8  
sending write request to follower server 0 (http://localhost:8080)  
[PASS] follower successfully handled request, in 0.162s  
[PASS] data consistency verified
```

21:49:47	SVR_00	[REQUEST]	Client GET Active auctions from /active
21:47:19	SVR_00	[REQUEST]	Batching Engine: Flushing 1 accumulated requests to Paxos InstanceBATCH
21:47:19	SVR_00	[REQUEST]	Leader received 122 bytes for batchingBATCH
21:47:19	SVR_00	[REQUEST]	Follower 0 forwarded request to Leader 8
21:47:19	SVR_00	[REQUEST]	Client POST /create , User: 1 , Item: ForwardedItem

כמו שראינו בסעיפים הקודמים, הfollowers בהצלחה שלח את הבקשה למנהיג. כלומר, אנו מקיימים את הדרישה שבה הלקוח יכול לפנות לכל שרת. ובעצם באופן כללי במערכת שלנו, השרתים שקופים ללקוח.

מבחן 9 – קריאה סדרתית

נבצע קריאה סדרתית במערכת.

```
[TEST]  
Testing sequential consistency of read  
tracking item 'seqItem_1985' across random servers  
[PASS] passed 50 operations and saved sequential consistency. final price seen: 10
```

נבצע 50 פעולות מעורבות במערכת, כתיבה וקריאה. לאחר ביצוע 50 הבקשות הללו, נבקש לראות מכרז מסוים, נראה שהוא לא יקבל ערך נמוך יותר מעצמו, כלומר מונוטוני. הבדיקה הזו מייצגת לקוח משתנה במהירות שמעדכן את הנתונים וקורא אותם מיד מנקודות קצה שונות בקלאסטר. נראה כי הבדיקה שלנו הצליחה והקריאה היא אכן סדרתית.

מבחן 10 – השלמת פעולות ה API במערכת – הוכחה שלהן

נראה את כלל הפעולות שניתן ונדרש לבצע ב API של המערכת :

```
[TEST]  
Testing API  
[PASS] API verified: getting value of multiple keys supported.  
[PASS] API verified: getting value of a single key supported  
[PASS] API Verified: updating a key, linearizable write supported  
[PASS] API verified: updating a list of keys (batch write) supported  
[PASS] API verified: read modify write - atomic logic check supported.
```

המבחן האחרון מוכיח בצורה מקיפה כי השרת מצליח לבצע סנכרון לפני ואחרי הקריאות ומוודא כי המידע עדכני עבור הלקוח, נראה מנגנון batch מקצב timestamps ייחודי עבור כל פקודה. הוא מבצע קריאות מפתח מרובים, ע"י חיפוש לפי שם והחזרת רשימת פריטים. הוא מבצע קריאות מפתח יחיד – אימות ID עבור מכרז. מבצע פעולת כתיבה ליניארזיבילית – מבצע הצעת מחיר למכרז. מבצע פעולת כתיבה של batch ליניארזיבילי. בודק את ה read modify write האטומי. בודק כי במידה ומגישים הצעה נמוכה יותר, המנגנון מבצע בדיקה לוגית אטומית ומגן על שלמות הנתונים ודוחה את הפקודה גם מתוך batch. נראה כי כלל המבחנים עברו בהצלחה והמערכת שלנו מכילה את כל הרכיבים הדרושים.

UI - Application

על מנת לבצע דמו לקוח אמיתי, לעקוב אחר המערכת בצורה מסודרת, תוך כדי צפייה במצב של המערכת, בנינו UI, לאחר הסתכלות על הדוגמא שניתנה עבור מערכת scooter. במערכת שלנו נוכל מצד אחד להראות דמו אמיתי ללקוח יחיד, מצד שני לשמש כבקרה עבור כלל הלקוחות, כולל דמו חיצוני של המבחן שהראינו בפרק הקודם. נתאר בקצרה את המערכת שבנינו.

הבחינה של המערכת מתרחשת בשתי אופנים, תחת ה: etc

- test.py – סקריפט פייתון המכיל את המבחנים עבור המערכת כפי בדקנו אותה.
- תיקיית frontend – בתוכה נמצאת תיקיית src המכילה את קוד המקור של האפליקציה:



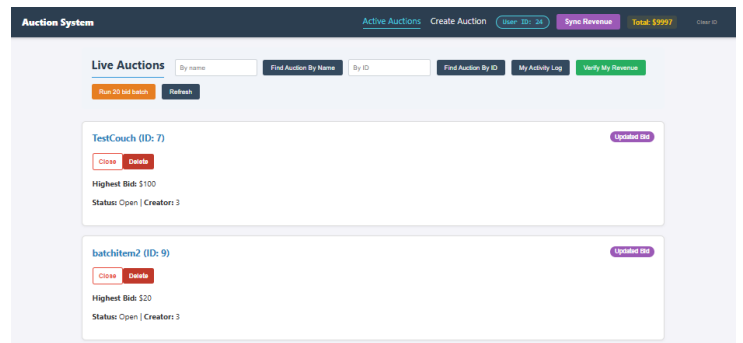
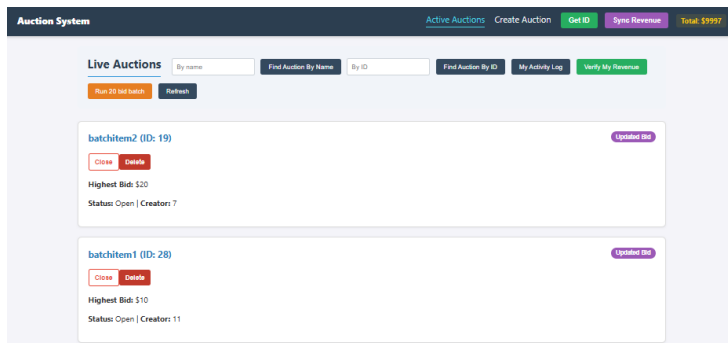
האפליקציה מחולקת אל הרכיבים הבאים:

שכבת התקשורת: auction.service.ts: הגשר בין הממשק לקלאסטר השרתים. מתבצע תקשורת HTTP מול Nginx Gateway בכתובת localhost/api ולא אל מול שרת ספציפי, על מנת לדמות את הגישה של הלקוח למערכת המבוזרת בצורה שקופה. שכבה זו משמשת שכבת הפשטה אל מול הלוגיקה של המערכת, היא משמשת כצינור המעביר בקשות HTTP ומחזיר אובייקטים מסוג observable. הוא פונה אל היעדים הייחודיים של המערכת, לפי קריאות ה api הנדרשות, כפי שקבענו ב auction_api.go.

שכבת האפליקציה הראשית app: משמשת כמעטפת של האפליקציה, מנהלת את staten המקומי של המשתמש, יחד עם אינדיקטורים ויזואליים, בזמן שממתין לתשובה מהמערכת. מעבר לצד הלקוח, הוא טוען את מערכת הניטור הוויזואלית לניטור צד השרתים.

המערכת שלנו מורכבת משלוש קומפוננטות:

1. ניהול המכירות auction-list: המסך המרכזי העליון, מציג את תמונת המצב של המערכת מהשרתים. שכבה זו נראית בצורה הבאה:



בממשק זה אנחנו מראים ביצוע פעולות קריאה ליניארזיביליות בעזרת: `sync`, `updated bid`, `revenue`, `verify my revenue` – קריאה של ההצעה הכי גבוה עבור מכרז מסוים, קריאה של סכום ההצעות שהתקבלו עבור כל המכרזים, קריאה של סכום הכסף שהשתמש הרוויח.

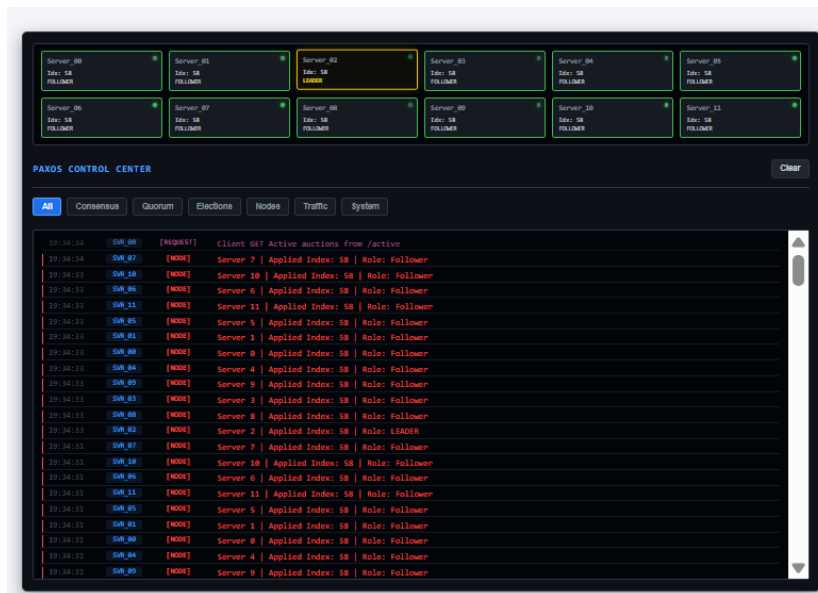
אנחנו מראים ביצוע קריאות סדרתיות בעזרת לחיצה על `Refresh` וצפייה בכל המכרזים, קריאה של מכרז לפי `id` שלו (קריאה יחידה) בעזרת `Find Auction By ID`, קריאה של המכרזים לפי השם שלהם (קריאה מרובה) בעזרת `Find Auction By Name`, קריאה של המכרזים של המשתמש בעזרת `My Auction Log`.

אנחנו מראים ביצוע של פעולות כתיבה ליניארזיביליות: המשתמש יכול לסגור או למחוק את המכרזים של עצמו (אם לא יהיו שלו לא יוכל לעשות זאת). משתמש יכול להירשם למערכת על ידי לחיצה על הכפתור `get ID`. בנוסף – עבור הוכחת המערכת הוספנו כפתור בשביל הוכחת היעילות של `batch` במערכת הלקוח יכול להריץ 20 הצעות על מכרזים.

2. יצירת מכרזים `create-auction`: מסך משני עליון. בעזרת רכיב זה אנחנו מייצגים ממשק עבור הוכחת יצירת משתמש של לקוח:

הרכיב משתמש בלוגיקת היצירה של מערכת `state machine` ומספק ממשק להרשמה, כך שהוא מושך באופן אוטומטי את `id` של `user` מה `local storage` ומצמיד אותו לבקשת יצירת המכרז.

3. מערכת הניטור והבקשה של המערכת `paxos-monitor`: הינו מייצג מרכז שליטה של המערכת המבוצרת של צד השרת ונועד לעזור לסדר את ההודעות המתקבלות מהשרת בצורה אינטואיטיבית על מנת לבחון את המערכת המבוצרת ולהוכיח את נכונותה. נמצא בחלק התחתון של הדף:



הרכיב מקשיב לכל הערצוים של השרתים הנמצאים בקלאסטר בצורה ספציפית, ע"י שימוש בeventsources נפרד. על חיבור מאזין לרצף system/logs של שרת ספציפי בקונטיינר ספציפי. כך שאנחנו יכולים להציג את ההודעות שהמערכת המבוזרת שלנו שומרת. אנחנו מסננים את ההודעות לפי הסוג שלהם ומכווינים אותם לחלק הנכון בטבלה. סוגי ההודעות שאנו מסננים: הודעות NODE – מצב שרת והאינדקס האחרון ששמר (מציג את snapshot האחרון ששמר). Consensus מציג את התהליך להגעה לקונצנזוס בכל פעולה שמתבצעת. Quorum מציג את ההגעה אל Quorum של שרתים עבור החלטה מסוימת. Elections מציג את המנהיג הנוכחי והמנהיג שניצח. Traffic מציג בקשות שהתקבלו מהלקוחות. System מציג הודעות מערכת. אנחנו משתמשים בwatchdog על מנת לבדוק מתי התקבל הקשר האחרון אל כל שרת, אם לא התקבל מידע שהשרת חי במשך 5 שניות מסמן אותו כoffline. השרת שנבחר להיות המנהיג מסומן בצהוב, על מנת לבחון מתי נהיה כשל במערכת. אנחנו משתמשים בfilter-online.pipe: נמצא תחת node-status-grid והינו צינור של angular לסינון תצוגת הצמתים בgrid והצגת המידע הרלוונטי בכל רגע.

ההודעות שמוצגות נשמרות ומעובדות במערכת המבוזרת בעזרת 2 קבצי קוד: logger.go תחת server/logger: קובץ זה מרכז את הנתונים מהמערכת המבוזרת. מחזיק rawevents, ערוץ המקבל את ההודעות מכל המערכת. Clients – מפה המחזיקה את הערוצים הפתוחים לכל דפדפן פתוח. אנחנו מריצים goroutine נפרד ומפיצים את ההודעות לכל הדפדפנים הפתוחים. אנחנו משתמשים בselect כדי שמידע לא יתעכב לשאר הלקוחות אם אחד מהם איטי.

logger_filter.go תחת server/api: נקודת ההקצה המטפלת בבקשות של system/logs/. משנה את headers אל event stream ומגדירה חיבור פתוח לאורך זמן, בכל חיבור חדש הפונקציה יוצרת ערוץ ייעודי ורושמת אותו בeventhub. כאשר לקוח מתנתק – מסירה אותו אוטומטית.

סיכום ומסקנות

בפרויקט זה תכננו, מימשנו ובדקנו מערכת מכרזים מבוססת על אלגוריתם Multi Paxos. בנינו את המערכת תחת אפיון ספציפי, תוך עמידה דרישות של עקביות, זמינות ועמידה בפני כשלים ותקלות.

הישגים עיקריים של המערכת:

- ארכיטקטורה רובסטית וגמישה: בנינו את המערכת בצורת `microservices` על גבי `docker` `containers` עם 12 צמתים. השתמשנו ב`etcd` עבור מנגנוני כשל, בחירת המנהיג ו`membership`. שימוש ב`etcd` עזר לנו להתמודד עם מצבי קיצון והבטיחה `Liveness` גבוה של המערכת.
- אופטימיזציות ביצועים: מימשנו מנגנונים נוספים בפרוטוקול `paxos` על מנת לשפר את ביצועי המערכת:
- `Adaptive Quorum`: מנגנון ייחודי המשתמש בסוגי `quorum` שונים. עבור מספר צמתים גבוה השתמשנו ב`grid quorum`.
- `Batching and pipelining`: אפשרו למערכת לתמוך ב`throughput` גבוה, תוך שמירה על ליניאריות המערכת.
- `Stable leader`: דילוג על שלב `prepare` כאשר המנהיג יציב צמצם את `latency` של המערכת.
- עמידות המערכת והתאוששות של `nodes`: שילוב מנגנון `WAL`, `snapshots` הבטיחו כי המערכת תעמוד בפני קריסות חומרה. הוכחנו באמצעות מבחני `recovery` כי צומת שנופל מסוגל להשתקם, בין עם טעינה מהדיסק ובין עם סנכרון מקומי מול שאר הקלאסטר, ללא פגיעה בפעילות המערכת.
- בחינת המערכת: ביצענו מבחנים רבים למערכת, שעברו, כדי להוכיח את עמידת המערכת בדרישותיה.
- שקיפות וניטור: פיתחנו `frontend` שדרכו יכולנו לפקח על הנכונות של המערכת בצורה יעילה יותר.
- ליניארזיביליות: מימשנו כתיבה וקריאה שעומד בדרישות הליניארזיביליות.
- בנינו מערכת `state machine` עם רעיון מקורי של מכרז.

מסקנות:

בפרויקט זה למדנו לבנות מערכת מבוססת אמיתית. במהלך הפרויקט למדנו להשתמש עם כלים חדשים רבים: כתיבה ב`go`, שימוש ב`docker`, שימוש ב`etcd` ובניית מערכת והחלטות ארכיטקטורה שיעלנו בכל שלב. חשבנו ובנינו בעצמנו את מכונת המצב ותהינו לגבי הקריאות והחשיבות שלהן והשיוך אל סוג הקריאה. למדנו על `trade-off` סין `latency` אל העקביות של המערכת. למדנו על מנגנוני סנכרון נדרשים עבור המערכת. טיפלנו במקרי קצה ובחנו אותם, ולמדנו על ההשפעה של שינויים טופולוגיים במערכת.